MAJOR PROJECT · CONCEPT & DESIGN DOCUMENT

Sentra: A Unified Uptime & Reliability Monitoring Platform

How every part of the system works, explained from the ground up — assuming no prior backend experience.

Document 1 of 2 — Concepts, Architecture & Viva Preparation

Companion document — Phase-wise Implementation Plan

Stack — Java 21 · Spring Boot 4.1 · PostgreSQL · React

Prepared for — Kshitij, Swami Keshvanand Institute of Technology

Date — 22 September 2026

Read this document first, then the build plan

Contents

0. How to Read This Document

What this document assumes · How it is organised

1. Backend Fundamentals You Need First

1.1 What a “backend” actually is

1.2 The request–response cycle

1.3 API, endpoint, route — the same idea, three names

1.4 HTTP in five minutes

1.5 Why a database and not just files

1.6 The three layers: Controller, Service, Repository

1.7 What Spring Boot actually does for you

1.8 The unusual part: code that runs when nobody is there

2. The Problem This Platform Solves

2.1 Three failures that happen constantly

2.2 The insight that turns seven tools into one platform

2.3 Tools that already exist, and why you are still building this

3. The Architecture

3.1 The whole system on one page

3.2 Every component and what it is responsible for

3.3 The data model in plain English

3.4 Entity relationships

3.5 Why one Check table instead of seven

4. How the Connection Is Built: Push vs Pull

4.1 The two possible directions

4.2 Pull — your server dials out

4.3 Push — their server calls you

4.4 Why there is no SSH, no agent and no config file

4.5 Which of your features uses which model

5. The Seven Features, One by One

5.1 Heartbeat monitoring (cron / dead man’s switch)

5.2 HTTP uptime monitoring

5.3 TLS certificate expiry

5.4 Domain expiry

5.5 TCP port reachability

5.6 Multi-step synthetic transaction

5.7 Deploy timeline correlation

6. The Shared Engine — Where the Real Engineering Is

6.1 The scheduler, and why it is one scanner

6.2 The life of a single check, step by step

6.3 The incident state machine

6.4 Hysteresis: not panicking over one blip

6.5 Alerting: deduplication, backoff, escalation

6.6 Idempotency: the same event arriving twice

6.7 How uptime percentage is calculated

6.8 The public status page

7. Security and Multi-Tenancy

7.1 Two completely different kinds of authentication

7.2 Multi-tenancy: keeping users out of each other's data

7.3 SSRF — the most important security problem in this project

7.4 Protecting the heartbeat token

7.5 Verifying the deploy webhook

8. Data Lifecycle — How Fast Rows Pile Up

8.1 The arithmetic · 8.2 Retention and rollups

9. How the React Frontend Talks to Spring Boot

10. What "24×7" Actually Requires

11. Glossary

12. Viva Questions, Rated by Depth

12.1 How the ratings work

12.2 Level 1 — Basics (rating 1–3)

12.3 Level 2 — Design understanding (rating 4–5)

12.4 Level 3 — Engineering judgement (rating 6–7)

12.5 Level 4 — Deep / research-grade (rating 8–10)

12.6 The five questions you must not fumble

0. How to Read This Document

Five minutes of orientation before the real content starts.

This document explains **what your project is, how each piece works, and why each piece exists**. Its companion document explains **how to build it, in order, week by week**. Read this one first — the build plan assumes you already understand the vocabulary introduced here.

0.1 What this document assumes

It assumes you can write Java, and that you have seen a Spring Boot application before but have not built a substantial one. It does **not** assume you know what a scheduler, a state machine, idempotency, SSRF, or multi-tenancy are. Every one of those is introduced from zero, in the place where it first matters.

Wherever a term appears for the first time in bold, it is also defined in the Glossary (Part 11), so you can look it up again later without hunting.

0.2 How it is organised

- **Parts 1–2** build the foundation: backend basics, then the real-world problem.
- **Parts 3–4** answer your architectural question directly — how the platform connects to the things it monitors.
- **Part 5** walks through each of your seven features and answers, for every one: *why is it needed*, *how does it connect*, *how does it fetch the information*, *what gets stored*, and *what goes wrong*.
- **Parts 6–8** cover the shared machinery underneath — this is the part examiners probe hardest, because it is the part that is genuinely engineering rather than CRUD.
- **Parts 9–11** are practical: frontend, deployment, vocabulary.
- **Part 12** is exam preparation: forty-three questions a teacher could ask, each rated for depth out of 10, each with the answer you should give.

> **ONE SENTENCE TO REMEMBER**
>
> Every feature in this project is the same idea wearing a different costume: **something must prove it is healthy inside a time window, and if it fails to prove it, people get told.** Once you see that, the whole architecture stops being seven projects and becomes one.

1. Backend Fundamentals You Need First

If you are already comfortable with controllers, services and repositories, skim to §1.8 — that section contains the one idea in this project that is genuinely new.

1.1 What a “backend” actually is

A backend is **a program that runs on a computer you control, waits for messages, and answers them**. That is the entire definition. It is not a special kind of software; it is an ordinary program whose main loop is “listen, receive, respond.”

When you open a website, your browser is a **client**. It sends a message over the internet asking for something. Somewhere, a machine is running a program that is permanently awake, listening on a network port. That program is the **server**. It reads the request, decides what to do, usually reads or writes a database, and sends a reply.

The frontend (React, in your case) is only the part the human looks at. It holds no permanent data. Refresh the page and everything it was holding in memory disappears. The backend is where data actually lives, where rules are enforced, and where work that must happen reliably is done.

> ANALOGY
>
> Think of a restaurant. The **frontend** is the menu and the dining room — it is designed for the customer and it looks nice. The **backend** is the kitchen: the customer never sees it, it holds all the ingredients, and it is where anything real actually happens. The **API** is the waiter — a fixed, agreed set of things you are allowed to ask for, in a format the kitchen understands.

1.2 The request–response cycle

Nearly everything in web development is this loop repeated at speed:



The server is awake the whole time. The client only appears when a human does something.

Figure 1.1 — The request–response cycle

Note carefully: **the client starts the conversation**. The server does not phone the browser. This is the default assumption of web development, and Part 4 is entirely about the fact that your project breaks it — deliberately, in two different ways.

1.3 API, endpoint, route — the same idea, three names

An **API** (Application Programming Interface) is the published list of things a program will do if you ask correctly. When your backend exposes an API, it is announcing: "here are the exact addresses you may send messages to, here is what each one expects, here is what it will give back."

Each individual address is an **endpoint** (also called a **route**). An endpoint is the combination of an HTTP method and a path:

```
GET    /api/checks          ← list all my checks
POST   /api/checks          ← create a new check
GET    /api/checks/42       ← fetch check number 42
DELETE /api/checks/42       ← delete check number 42
```

Notice that `/api/checks` appears twice with two different methods and two completely different meanings. The method is part of the identity of the endpoint, not decoration.

1.4 HTTP in five minutes

HTTP is the language clients and servers speak. A request has four parts.

Part	What it is	Example from your project
Method	The verb — what kind of action you intend	`POST` (create something)
Path	The address of the thing you are acting on	`/api/v1/checks`
Headers	Metadata: who you are, what format you are sending	`Authorization: Bearer eyJhb...`
Body	The actual data, usually JSON. Absent on GET.	`{"name":"API uptime","url":"https://..."}`

The reply comes back with a **status code** — a three-digit number that tells the client, at a glance, what happened. You only need to know the families plus a handful of specific values:

Code	Meaning	Where it shows up in your project
200	OK	A monitored site is healthy; your API returned data successfully
201	Created	A new check was created
204	No content	A heartbeat ping was accepted — nothing to return
400	Bad request	The user submitted a check with no URL
401	Unauthenticated	Missing or expired login token
403	Forbidden	Logged in, but trying to read *someone else's* check
404	Not found	Check ID does not exist
409	Conflict	A check with that name already exists in the project
429	Too many requests	Someone is hammering your heartbeat endpoint
500	Server error	Your own code threw an exception
502/503	Bad gateway / unavailable	A monitored site is down — the most common failure you will record

> WORTH INTERNALISING
>
> 401 means *"I don't know who you are."* 403 means *"I know exactly who you are, and you are not allowed."* Mixing these two up is one of the most common mistakes in student projects, and one of the easiest things for an examiner to catch.

1.5 Why a database and not just files

You could save monitoring results into a text file. It would work for a week. Then it would fail, for four reasons that matter here:

- **Concurrency.** Your scheduler will finish several probes at the same instant. Two threads appending to one file corrupt each other. A database handles simultaneous writes correctly by design.
- **Querying.** You will constantly ask questions like "every failed run for check 42 in the last 7 days." In a file that means reading the whole file. In a database with an index it is near-instant, however large the table grows.
- **Transactions.** Opening an incident means writing several rows that must either all succeed or all fail. A database gives you that guarantee; a file does not.
- **Durability across restarts and deploys.** Hosting platforms give a web service an **ephemeral filesystem** — files written by the app are wiped on every restart. A managed database survives.

You will use **PostgreSQL**. Use it from day one, including on your laptop — developing against one database and deploying to another is a reliable way to discover a bug the night before your demo.

1.6 The three layers: Controller, Service, Repository

Spring Boot applications are conventionally split into three layers. The split exists so that each piece of code has exactly one reason to change.

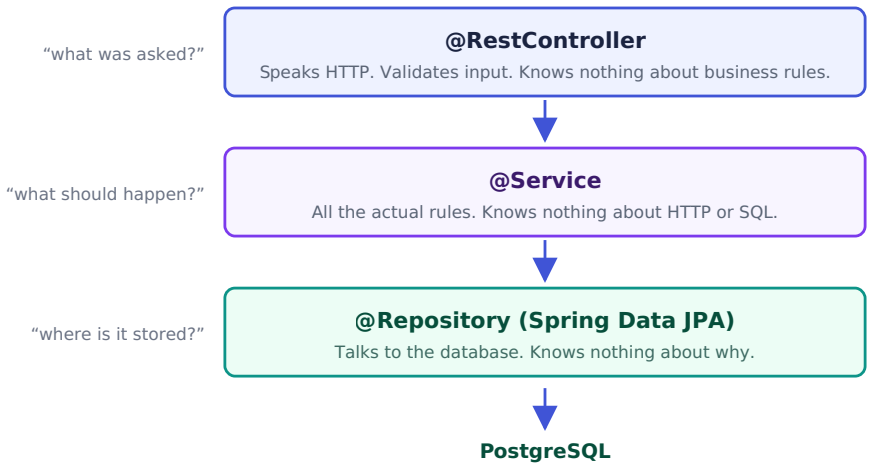


Figure 1.2 — The standard three-layer split. Dependencies only ever point downward.

Concretely, in your project, creating a check looks like this:

```
@RestController
@RequestMapping("/api/v1/checks")
class CheckController {
    private final CheckService service;   // injected by Spring

    @PostMapping
    ResponseEntity<CheckResponse> create(@Valid @RequestBody CreateCheckRequest req,
                                         @AuthenticationPrincipal AppUser user) {
        // The controller does THREE things and no more:
        //   1. accept the HTTP request  2. hand it to the service  3. shape the reply
        CheckResponse created = service.create(req, user);
        return ResponseEntity.status(HttpStatus.CREATED).body(created);
    }
}
```

All the interesting decisions — is this URL allowed, what interval is legal, when should the first probe run — live in `CheckService`. That matters because the service can then be tested without starting a web server, and reused by a scheduled job that has no HTTP request at all. You will do exactly that.

1.7 What Spring Boot actually does for you

Students often use Spring Boot without being able to say what it contributes. Here is the honest list, and an examiner may well ask.

It gives you	What you would otherwise write yourself
An embedded web server	Tomcat is bundled inside your JAR. No separate server to install, configure or deploy to. `java -jar app.jar` and you are serving HTTP.
Dependency injection	You declare that `CheckService` needs a `CheckRepository`; Spring constructs and wires them. You never write `new` for your own components, which is what makes them swappable in tests.
Auto-configuration	Put a PostgreSQL driver on the classpath and a URL in config, and Spring builds a connection pool, a transaction manager and an `EntityManager` for you.
Spring Data JPA	You declare `List<Check> findByEnabledTrueAndNextRunAtBefore(Instant t)` as an interface method with no body, and Spring generates the SQL.
Spring Security	A filter chain for authentication and authorisation, so you are not hand-rolling password hashing and token parsing.
Scheduling and async	`@Scheduled` and `@Async` — background work without managing raw threads. This is the beating heart of your project.
Actuator	Ready-made `/actuator/health` and metrics endpoints. Pleasingly, your monitoring platform can monitor itself with these.

1.8 The unusual part: code that runs when nobody is there

This is the single most important idea in your project, and it is where it departs from every CRUD application you have written.

In a normal web application, **all code is triggered by a user**. Nobody clicks, nothing runs. The application is, in effect, asleep between requests.

A monitoring platform cannot work that way. If your platform only checked a website when someone opened the dashboard, it would be a manual refresh button, not a monitor. The entire value is that it watches at 3 a.m. when everyone is asleep and no browser is open anywhere on earth.

So your backend has **two independent sources of activity**:

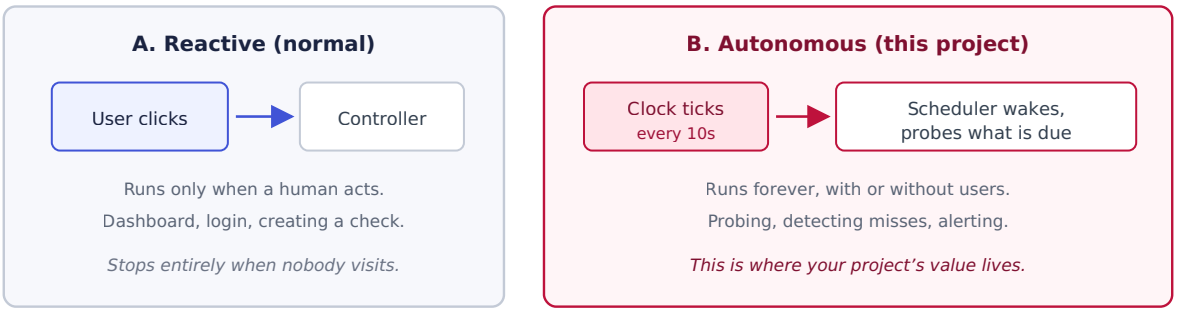


Figure 1.3 — Two sources of activity. Most student projects only ever have column A.

The practical consequences of column B run through the entire rest of this document:

- The dashboard becomes a **window onto results that already happened**, not the thing that causes them. Opening the dashboard triggers no probes.
- Your application must be **deployed somewhere that stays awake**. Part 10 deals with this, and it is a real trap on free hosting tiers.
- Errors in background code have **nobody to show a 500 page to**. If a probe thread dies silently, monitoring stops and nothing appears broken. Background code needs deliberate logging and its own failure handling.

2. The Problem This Platform Solves

An examiner's first real question is almost always "why does this need to exist?" This part is your answer.

2.1 Three failures that happen constantly

Failure one — the backup that stopped running

A team sets up a nightly database backup at 2 a.m. using cron. It works. Six months later, a disk fills up and the script begins exiting early. It still runs, it still logs, it just no longer produces a backup. **Nothing raises an alarm, because nothing crashed.** The failure is discovered eleven weeks later, at the worst possible moment: when somebody needs to restore.

The essential problem here is that **you cannot detect the absence of an event by watching for an event**. A job that fails loudly is easy. A job that silently stops running produces no signal at all — and silence is indistinguishable from success unless something is actively expecting the job to report in.

Failure two — the certificate nobody renewed

TLS certificates expire, typically every 90 days with automated issuers. When one expires, every visitor gets a full-page browser security warning. The site is technically up — the server responds fine — but no customer will proceed past that screen. Teams routinely find out from an angry customer, not from their own systems, because a certificate expiry is a **future-dated failure**: the thing that will break is perfectly healthy today.

Failure three — "it returns 200, so it's fine"

A checkout page loads. The homepage loads. Every individual endpoint returns HTTP 200. But the payment step has been broken since this morning's deploy, because a configuration value was renamed. Every simple uptime monitor on earth reports this site as 100% up, and it is losing money every minute.

Checking that a server responds is not the same as checking that it **works**. That gap is exactly what your synthetic transaction feature closes.

> **THE THREAD CONNECTING ALL THREE**
>
> In each case, something needed to be **continuously verified by an outside observer**, and no such observer existed. Nobody was negligent; the information simply was not being collected by anything.

2.2 The insight that turns seven tools into one platform

At first glance your seven features look unrelated. Watching a cron job, pinging a website, reading a certificate date, checking a WHOIS record, opening a TCP socket, replaying a login flow, recording a deploy — seven different problems.

They are not. Six of them are the same sentence with one word changed:

THE UNIFYING ABSTRACTION

A check is a thing that must prove it is healthy within a time window. If it fails to prove it, an incident opens and people are told.

Everything that differs between your features is contained in one single method: *how does this particular check prove health?*

Check type	How it proves it is healthy
Heartbeat	It calls you before its deadline passes
HTTP uptime	It answers your call with the status code you expected
TLS certificate	Its expiry date is still comfortably in the future
Domain	Its registration expiry date is still comfortably in the future
TCP port	It accepts a socket connection
Synthetic transaction	A whole sequence of steps completes with every assertion passing

Deploy timeline correlation is the odd one out, and deliberately so: it is not a check at all, it is **context**. It exists to answer the question everybody asks thirty seconds after an alert fires — *"did we just change something?"*

This is what justifies calling the project a platform rather than a collection. You write the scheduler once, the incident engine once, the alerting pipeline once, the dashboard once. Adding a seventh check type then costs you one class implementing one interface. **That is the architectural claim your project makes, and demonstrating it is how you prove the design is sound.**

2.3 Tools that already exist, and why you are still building this

Be honest about this in your report — claiming you invented monitoring would be worse than useless. Commercial equivalents exist: Healthchecks.io for heartbeats, UptimeRobot and Pingdom for HTTP uptime, PagerDuty for on-call alerting, Datadog and Grafana Cloud for everything at enterprise scale.

Three defensible reasons to build it anyway:

- **They are separate products.** A small team ends up with four subscriptions, four dashboards and four alert inboxes, and no single view that says "here is everything about the health of my system, plus what we changed today." Unification is the actual product idea.
- **Cost and control at small scale.** Free tiers cap out quickly and prices assume a funded company. A self-hostable tool that a two-person team runs on one small server is a real niche.
- **The deploy-correlation view is genuinely uncommon** at the small end of the market. Enterprise tools have it; cheap uptime monitors do not.

The academic justification is separate and equally valid: the project exercises scheduling, concurrency, state machines, security, multi-tenancy and data lifecycle management in one coherent system. That is a curriculum, not a CRUD form.

3. The Architecture

What the pieces are, what each is responsible for, and how they fit together.

3.1 The whole system on one page

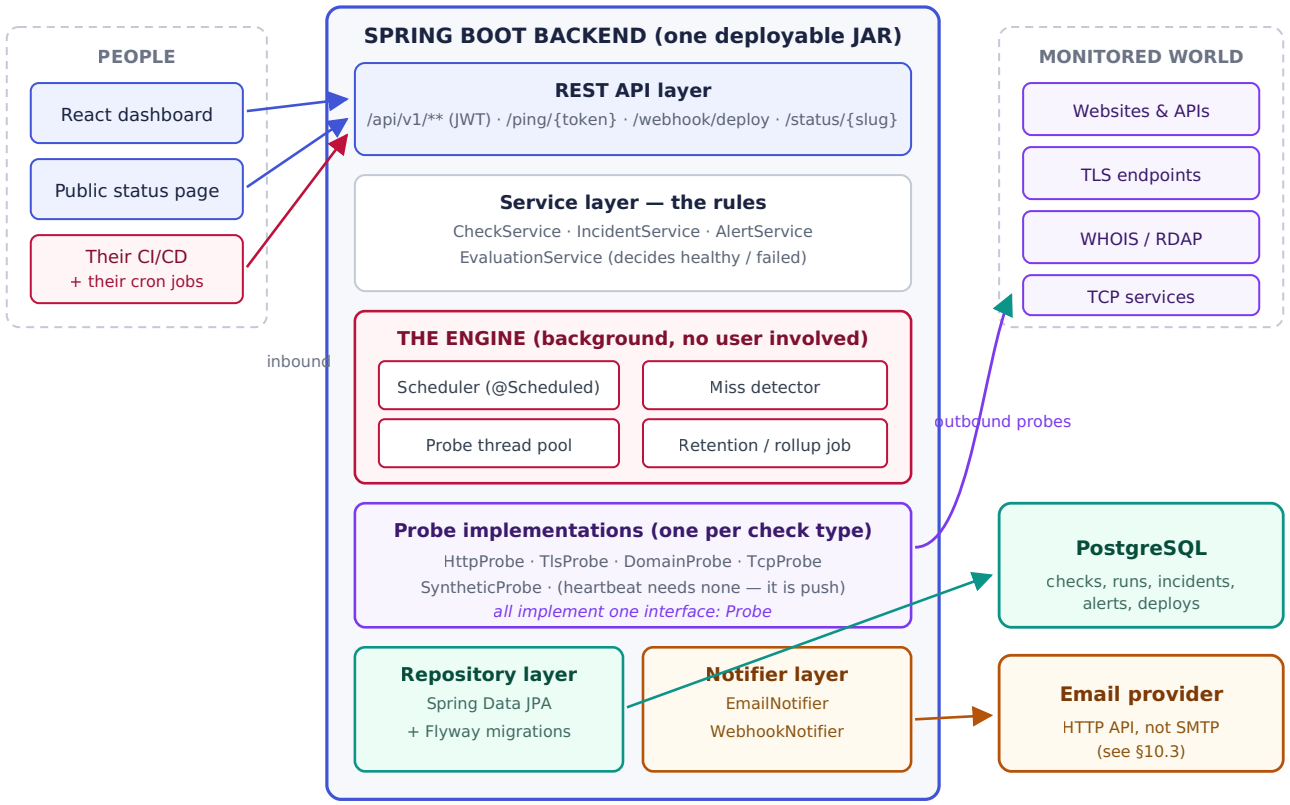


Figure 3.1 — The complete system. Note that traffic crosses the backend boundary in both directions.

3.2 Every component and what it is responsible for

Component	Responsibility	Why it is separate
REST API layer	Accepts HTTP from the dashboard, from monitored cron jobs, and from CI systems	Three audiences with three different auth models — keeping them as distinct controllers keeps the security config comprehensible
Service layer	All business rules: what intervals are legal, when an incident opens, who gets alerted	Must be callable both from a controller and from the scheduler, so it cannot depend on HTTP
Scheduler	Wakes on a fixed tick, finds checks whose `next_run_at` has passed, dispatches them	One central clock is far easier to reason about (and to scale) than thousands of individual timers

Component	Responsibility	Why it is separate
Miss detector	Finds heartbeat checks whose deadline passed with no ping	Detecting *absence* is fundamentally different from running a probe, so it is its own sweep
Probe thread pool	Actually performs network I/O, in parallel, off the scheduler thread	A single slow probe must never delay every other check on the platform
Probe implementations	One class per check type, each implementing the same `Probe` interface	This is the extension point — a new check type is a new class and nothing else changes
Incident engine	Turns a stream of pass/fail results into opened and closed incidents	Contains the hysteresis logic; without it your platform alerts on every momentary blip
Notifier layer	Sends the alert through whichever channels are configured	Strategy pattern, so adding a channel does not touch incident logic
Retention job	Deletes or aggregates old run rows nightly	Without it the database grows without limit — see Part 8
Repository layer	Database access via Spring Data JPA, schema managed by Flyway	Migrations mean your database schema is version-controlled alongside your code

3.3 The data model in plain English

Seven tables carry the whole system. Read this list slowly — if you understand these seven sentences you understand the project.

Table	What one row means
app_user	One person who can log in.
project	One logical grouping of checks owned by a user — e.g. “Urbnskn production”. Every check belongs to exactly one project. This is the unit of multi-tenancy and the unit a status page is built from.
check	One thing being watched. Holds the **shared** fields every check type needs (name, type, interval, enabled, next run time, current state) plus a JSON column holding the fields that only that type needs (a URL, a port, a list of synthetic steps).
check_run	One evaluation of one check at one moment: passed or failed, how long it took, what the response was, why it failed. This is by far your largest table.
incident	One continuous period during which a check was unhealthy. Opens when failures cross the threshold, closes when health returns. An incident groups many failing runs into one human-meaningful event — this is what stops your alerts becoming spam.
alert_rule / **alert_delivery**	The rule says *who to tell*, *through what channel*, *after how long*. A delivery row records one actual notification that was sent, so you can prove what was sent and avoid sending it twice.

Table	What one row means
deploy_event	One deployment that a CI system reported: when, which commit, which environment. Not a check — purely context to overlay on the timeline.

3.4 Entity relationships

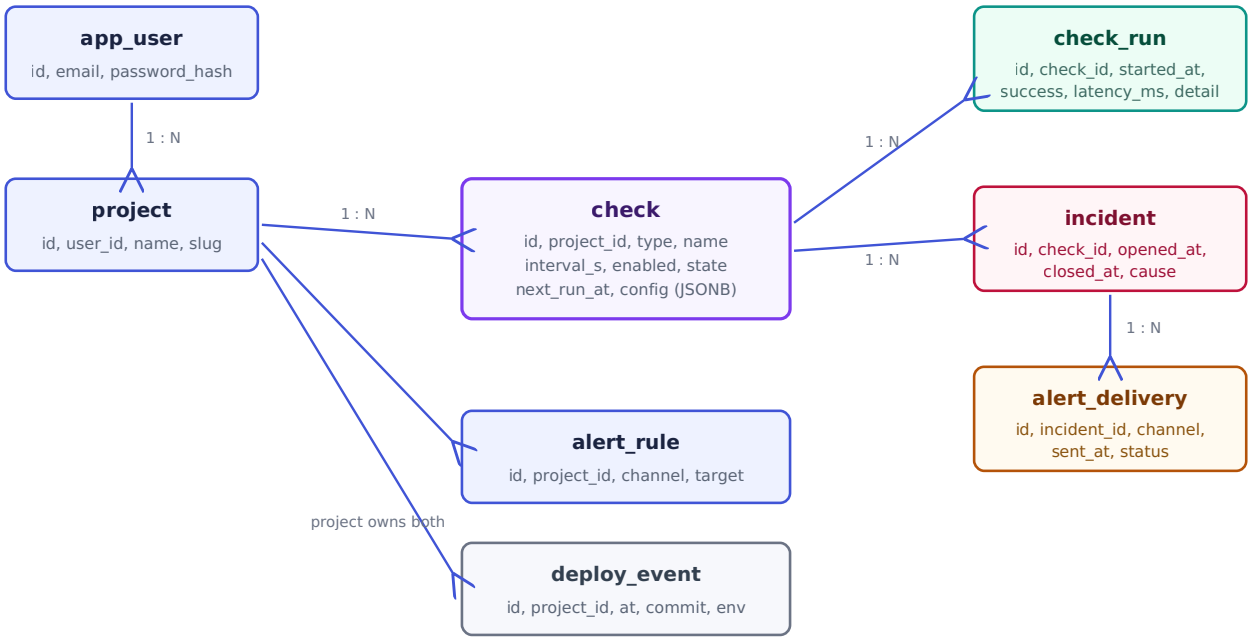


Figure 3.2 — Entity relationships. Everything hangs off `project`, which is what makes tenant isolation enforceable in one place.

3.5 Why one `check` table instead of seven

You have a genuine design choice here, and being able to defend it is worth marks.

Option A — a table per check type

`http_check`, `tls_check`, `tcp_check`… Each gets exactly the columns it needs, so the schema is strict and self-documenting. The fatal problem: your scheduler now has to query seven tables to ask "what is due?", your incident table needs seven nullable foreign keys, and adding a check type means changing the scheduler, the incident engine and the dashboard. The abstraction you claimed in §2.2 is not actually expressed anywhere in the code.

Option B — one table, shared columns plus a JSON column (recommended)

One `check` table holds everything common: id, project, type, name, interval, enabled, state, next_run_at. A single `JSONB` column called `config` holds the type-specific settings.

```
-- an HTTP check's config
{ "url": "https://urbnskn.in/health", "expectedStatus": 200, "timeoutMs": 5000 }

-- a TCP check's config
{ "host": "db.internal", "port": 5432, "timeoutMs": 3000 }

-- a TLS check's config
{ "host": "naamiofficial.in", "port": 443, "warnDays": [30, 14, 7, 1] }
```

Now the scheduler asks one table one question. The incident table has one foreign key. Adding a check type means writing one probe class and one validation rule — no schema change at all.

> **BE READY FOR THE OBVIOUS OBJECTION**
>
> The examiner may say: *"JSON columns mean the database cannot validate your data."* That is correct, and your answer is that validation moves into the application: each check type has its own config class with Bean Validation annotations, and the config is parsed and validated on write. You accept weaker database-level guarantees in exchange for an extension point — and you make that trade knowingly, which is the point.

In Java this is expressed with an enum for the type and a mapped JSON column:

```
@Entity @Table(name = "checks")
public class Check {
    @Id @GeneratedValue private Long id;

    @Enumerated(EnumType.STRING)
    private CheckType type;        // HEARTBEAT, HTTP, TLS, DOMAIN, TCP, SYNTHETIC

    private Integer intervalSeconds;
    private Integer gracePeriodSeconds;
    private Instant nextRunAt;        // the single most important column in the schema
    private boolean enabled;

    @Enumerated(EnumType.STRING)
    private CheckState state;      // HEALTHY, DEGRADED, DOWN, PAUSED

    @JdbcTypeCode(SqlTypes.JSON)
    private Map<String, Object> config;   // type-specific settings
}
```

And the extension point itself — the interface every probe implements — is this small:

```
public interface Probe {
    CheckType supports();                    // which type am I for?
    ProbeResult execute(Check check);        // go and find out if it is healthy
}

public record ProbeResult(
    boolean success,
    long latencyMs,
    String detail,          // "HTTP 200 in 340ms" or "connection refused"
    Integer daysRemaining   // only meaningful for TLS and domain checks
) {}
```

Spring will inject every implementation of `Probe` into a list automatically. You index them by type once at startup, and the dispatcher becomes a single map lookup with no `if/else` chain anywhere. **When an examiner asks how extensible your design is, this is the code to show them.**

4. How the Connection Is Built: Push vs Pull

This answers the question directly: there is no SSH, no agent to install, and no file to copy.

4.1 The two possible directions

Any monitoring system has to get information from somewhere else to itself. There are only two ways for that to happen, and the difference is simply **who starts the conversation**.

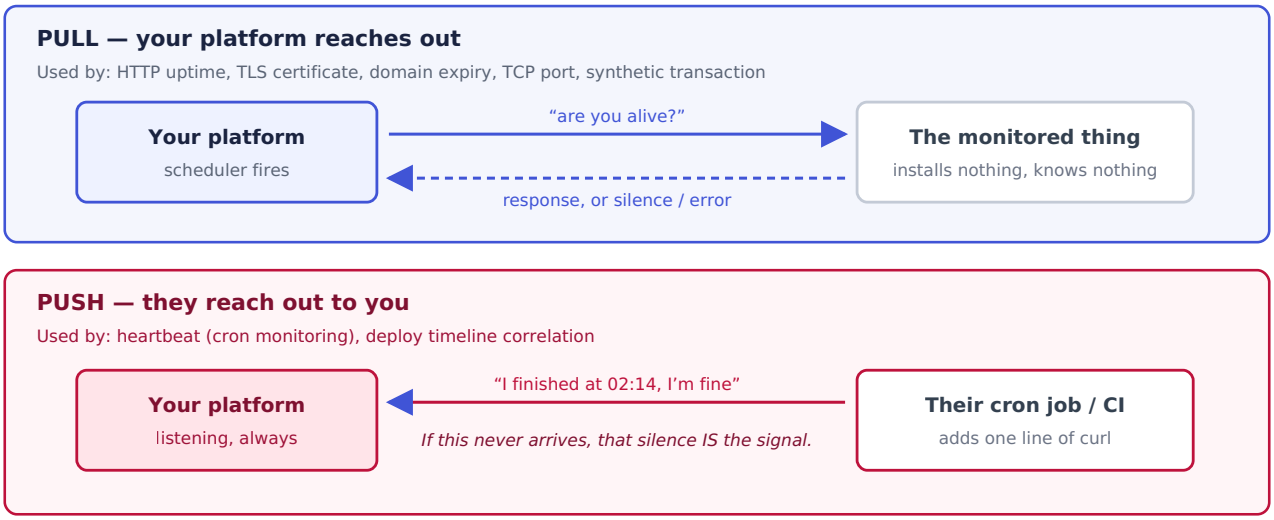


Figure 4.1 — The only two connection models, and which of your features uses each.

> ANALOGY
>
> **Pull** is ringing your elderly neighbour every morning to check she is fine. You initiate; she needs to do nothing except pick up.
>
> **Push** is agreeing that she will text you every morning by 9 a.m. Now *her silence* is the alarm — and crucially, this works even when she has no phone signal to answer your call, which is precisely the situation where you most want to know.

4.2 Pull — your server dials out

For five of your seven features, there is **nothing to set up on the monitored side at all**. The user types a URL or a hostname into a form on your dashboard, and you store it. From then on, your scheduler makes outbound network connections on its own timetable.

From your backend's point of view, a pull check is exactly the sequence in Figure 1.1, but with your backend playing the role of the *client* rather than the server — and with a website somewhere in the world as the server. Your platform is simultaneously a server (to your dashboard) and a client (to everything it monitors). That dual role is worth stating out loud in a viva; it is a point many students miss.

The user provides	Your backend does	Result recorded
A URL, e.g. `https://urbnskn.in/health`	Sends an HTTP GET with a timeout	Status code, response time, or the reason it failed
A hostname, e.g. `naamiofficial.in`	Opens a TLS connection, reads the certificate, stops	Days until expiry
A domain name	Queries RDAP / WHOIS over the network	Registration expiry date
Host and port, e.g. `db.example.com:5432`	Attempts a TCP socket connection	Connected, refused, or timed out
A list of steps with assertions	Executes them in order, carrying state between them	Which step failed, and why

The setup effort for the user is therefore **typing into a form**. That is not an oversimplification; that is genuinely all UptimeRobot and Pingdom require of you too.

4.3 Push — their server calls you

Push is needed exactly when the thing you care about **cannot be observed from outside**. You cannot look at a server from the internet and tell whether its backup script ran at 2 a.m. There is no port to knock on. The only entity that knows is the script itself, so the script has to tell you.

When the user creates a heartbeat check, your backend generates a random unguessable token and stores it against that check. The dashboard shows the user a URL:

```
https://sentra.app/ping/9f3c1e77-4a2b-4f9e-b3c7-8d1a55e0c412
```

The user adds **one line** to the job they already run:

```
# their existing crontab entry, before
0 2 * * *  /usr/local/bin/backup.sh

# after — the only change is what follows &&
0 2 * * *  /usr/local/bin/backup.sh && curl -fsS --retry 3 https://sentra.app/ping/9f3c1e77-...
```

Three details in that line matter, and each is a small piece of design you can defend:

- `&&` means the ping only fires **if the backup actually succeeded**. Using `;` instead would report success even when the script failed — a subtle and very common mistake.
- `-f` makes curl fail on HTTP errors, `-sS` keeps it quiet but still shows real errors, so a broken ping shows up in cron's mail rather than vanishing.
- `--retry 3` tolerates a momentary network blip, so you do not raise a false alarm because of one dropped packet. This is also exactly *why* your endpoint must be idempotent (§6.6) — the same ping may legitimately arrive more than once.

The deploy webhook works identically, except the CI pipeline sends a POST with a small JSON body instead of a bare GET:

```
# final step of a GitHub Actions / Jenkins pipeline
curl -X POST https://sentra.app/webhook/deploy/<project-token> \
     -H "Content-Type: application/json" \
     -H "X-Sentra-Signature: sha256=..." \
     -d '{"commit":"a1b2c3d","environment":"production","actor":"kshitij"}'
```

4.4 Why there is no SSH, no agent and no config file

Tools like Nagios or Zabbix do install an agent on the monitored machine, and tools that read CPU or disk usage genuinely need one — that information exists only *inside* the machine and no amount of poking from outside will reveal it.

Your platform does **black-box monitoring**: it observes only what is visible from the outside, exactly as a real user would experience it. That means:

Approach	Needs on the monitored host	Your project
Black-box (external probes)	Nothing	Yes — all five pull checks
Push from the job itself	One line of curl	Yes — heartbeat and deploy webhook
Agent binary installed	A process to install, run and update	No — out of scope
SSH-based pulling	SSH credentials handed to your platform	No — and deliberately not

> SAY THIS IN YOUR VIVA
>
> "I chose black-box monitoring, so the platform never needs credentials for the systems it watches. If my platform were compromised, an attacker would gain a list of URLs — not SSH access to my users' servers. Storing other people's SSH keys would make my database the most valuable target on the network, and that risk buys me nothing my feature set needs."
>
> That is a **security-by-design argument**, and it turns what looks like a missing feature into a deliberate decision.

There is a genuine trade-off, and you should name it before the examiner does: black-box monitoring can tell you *that* something is broken but often not *why*. It sees a 503; it cannot see that the cause was memory exhaustion. Deploy correlation (§5.7) is your partial answer to that limitation — it cannot read the server's memory, but it can tell you a deploy happened four minutes before things broke, which in practice resolves a large share of real incidents.

4.5 Which of your features uses which model

Feature	Model	User setup effort	Who initiates the network connection
HTTP uptime	Pull	Type a URL	Your server → their site
TLS certificate expiry	Pull	Type a hostname	Your server → their TLS port
Domain expiry	Pull	Type a domain	Your server → RDAP/WHOIS registry
TCP port reachability	Pull	Type host and port	Your server → their port
Synthetic transaction	Pull	Define steps in the UI	Your server → their app, repeatedly
Heartbeat / cron	**Push**	Paste one curl line	Their job → your server
Deploy correlation	**Push**	Add one CI pipeline step	Their CI → your server

Five of seven require no integration whatsoever. Two require a single line. **Nothing requires SSH, an agent, a downloaded file, or access to their machine.**

5. The Seven Features, One by One

For each: why it is needed, how it connects, how it fetches its information, what gets stored, and what goes wrong.

5.1 Heartbeat monitoring (cron / dead man's switch)

Why it is needed

Scheduled background jobs — backups, report generation, data syncs, invoice emails — fail silently. Nobody is watching them because by definition nobody is present when they run. As established in §2.1, **the absence of an event produces no signal**, so no amount of watching from outside will detect it. This is the only feature in the platform that can detect something that simply stopped happening.

The name "dead man's switch" comes from the train driver's handle that must be held down continuously; release it — whether deliberately or by collapsing — and the brakes apply. Absence of the expected signal is itself the trigger.

How it connects

Push. The user creates a heartbeat check with an expected interval ("this should ping me every 24 hours") and a grace period ("but do not panic until it is 1 hour late"). Your platform generates a token and shows a URL. The user appends one `curl` to their existing job, as shown in §4.3.

How the information is fetched

It is not fetched. It **arrives**, and separately, a sweep detects when it fails to arrive. These are two independent pieces of code, and understanding why there must be two is the core of this feature.

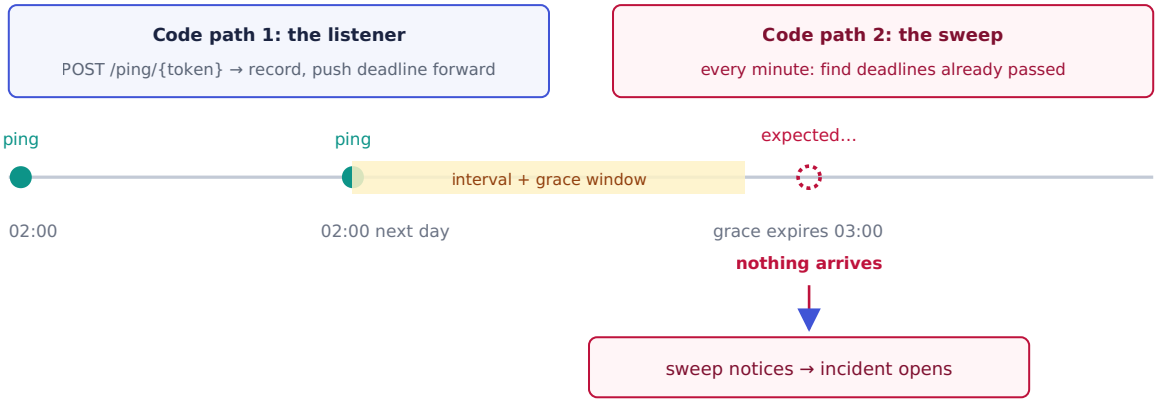


Figure 5.1 — Heartbeat monitoring needs two independent pieces of code: one that receives, one that notices nothing was received.

The listener is trivially simple:

```
@PostMapping("/ping/{token}")
ResponseEntity<Void> ping(@PathVariable String token) {
    heartbeatService.recordPing(token, Instant.now());
    return ResponseEntity.noContent().build();   // 204, nothing to say
}
```

And `recordPing` does three things: write a successful `check_run`, push `next_expected_at` forward to `now + interval + grace`, and close any open incident for this check. The sweep is the other half:

```
@Scheduled(fixedDelay = 60_000)
void detectMissedHeartbeats() {
    // "which heartbeats should have reported by now, and didn't?"
    List<Check> overdue = checkRepo.findOverdueHeartbeats(Instant.now());
    for (Check c : overdue) {
        evaluationService.recordFailure(c, "no ping received within interval + grace");
    }
}
```

What gets stored

- A `check_run` row per ping received (and per miss detected).
- `next_expected_at` on the check row — the single column the sweep query is indexed on.
- Optionally the ping's source IP and a client-supplied run duration, useful for the "this job is getting slower" trend.

What goes wrong

Problem	How you handle it
The job runs for three hours	A single end-of-job ping cannot tell you it started at all. Support an optional `/ping/{token}/start` call so you can measure duration and detect a job that began but never finished.
The job fails but still pings	That is the `;` versus `&&` mistake from §4.3. Also offer `/ping/{token}/fail` so a script can actively report failure instead of staying silent.
The same ping arrives twice	curl retried, or a proxy duplicated it. Handle with idempotency (§6.6) — a duplicate must not create a second run row.
Clock skew	Never trust a timestamp sent by the client. Record arrival time using *your* server clock, stored in UTC. Treat any client-supplied time as informational only.
A brand-new check has never pinged	"Never ran" is a genuinely different state from "was running, now stopped." Model it explicitly as `PENDING`, or you will alert the user the moment they create a check.
Someone guesses a ping URL	They could falsely mark a dead job as healthy. Hence a 128-bit random token, not a sequential ID (§7.4).

5.2 HTTP uptime monitoring

Why it is needed

It is the most obvious question anyone has about a website: *is it up?* Website downtime costs money directly, and the operator is usually the last to find out because they are not refreshing their own site every minute.

How it connects

Pull. The user types a URL. Nothing is installed anywhere.

How the information is fetched

Your backend makes an ordinary HTTP request, with deliberately strict timeouts, and judges the response. Spring Boot 4 ships `RestClient`, which is the simple synchronous choice:

```
@Component
class HttpProbe implements Probe {

    @Override public CheckType supports() { return CheckType.HTTP; }

    @Override public ProbeResult execute(Check check) {
        HttpConfig cfg = configMapper.read(check, HttpConfig.class);
        long start = System.nanoTime();
        try {
            ResponseEntity<String> res = restClient.method(cfg.method())
                    .uri(cfg.url())
                    .retrieve()
                    .toEntity(String.class);

            long ms = (System.nanoTime() - start) / 1_000_000;

            if (res.getStatusCode().value() != cfg.expectedStatus())
                return ProbeResult.fail(ms, "expected " + cfg.expectedStatus()
                                            + " but got " + res.getStatusCode().value());

            if (cfg.mustContain() != null && !res.getBody().contains(cfg.mustContain()))
                return ProbeResult.fail(ms, "body did not contain expected text");

            return ProbeResult.ok(ms, "HTTP " + res.getStatusCode().value());

        } catch (ResourceAccessException e) {      // timeout, DNS failure, refused
            return ProbeResult.fail(elapsed(start), classify(e));
        }
    }
}
```

> **THE DETAIL THAT SEPARATES A GOOD PROJECT FROM A BASIC ONE**
>
> There are **two different timeouts**, and you should configure both. A **connect timeout** caps how long you wait to establish the connection — exceeding it usually means the host is unreachable. A **read timeout** caps how long you wait for the response body after connecting — exceeding it usually means the server accepted you but is overloaded. Set connect to ~3 s and read to ~10 s. Without a read timeout, a single hanging site can occupy one of your probe threads indefinitely.

Distinguishing failure *kinds* is what makes your stored data useful. “Failed” is much less informative than:

Failure	What it usually means	Stored as
`UnknownHostException`	DNS is broken or the domain expired	`DNS_FAILURE`
`ConnectException` (refused)	Host is up, nothing listening on that port	`CONNECTION_REFUSED`
`SocketTimeoutException`	Firewall dropping, or server overloaded	`TIMEOUT`
`SSLHandshakeException`	Certificate expired or invalid	`TLS_ERROR`
Wrong status code	The app is running but broken	`UNEXPECTED_STATUS`

What gets stored

One `check_run` per probe: timestamp, success flag, latency in milliseconds, status code, failure category and a short human-readable detail. Never store the whole response body — truncate to a few hundred characters, or you will fill the disk with HTML.

What goes wrong

- **False positives from your own network.** If your hosting provider has a hiccup, every check fails at once and you alert every user about an outage that is yours. Mitigation: if more than, say, 80% of all checks fail in the same minute, suspect yourself and suppress alerts — a simple and very impressive safeguard to demonstrate.
- **Redirects.** Decide explicitly whether to follow them. Following hides the fact that a site is redirecting to an error page; not following turns a normal 301 into a false failure. Make it a per-check setting.
- **Being mistaken for an attacker.** Send an identifying `User-Agent` such as `Sentra-Monitor/1.0 (+https://sentra.app/bot)` so site owners can recognise you in their logs.
- **200 with an error page.** Exactly why the “body must contain” assertion exists — and why synthetic transactions (§5.6) exist as the fuller answer.

5.3 TLS certificate expiry

Why it is needed

An expired certificate does not make a site slow — it makes it unusable, because browsers show a full-page interstitial that most visitors will not click past. Certificates from automated issuers last 90 days, and the industry is moving toward even shorter lifetimes, so renewal automation failing quietly is a recurring real-world incident. The failure is **future-dated**: today everything is perfect, and on a specific date at a specific hour it breaks completely.

How it connects

Pull. The user gives a hostname (and optionally a port, defaulting to 443).

How the information is fetched

You open a TLS connection and read the certificate the server presents, then close immediately. You never send an HTTP request — the certificate is exchanged during the handshake, before any application data.

```
SSLSocketFactory factory = context.getSocketFactory();
try (SSLSocket socket = (SSLSocket) factory.createSocket()) {
    socket.connect(new InetSocketAddress(host, port), connectTimeoutMs);
    socket.setSoTimeout(readTimeoutMs);

    // SNI: essential. Without it, a shared host returns the WRONG certificate.
    SSLParameters params = socket.getSSLParameters();
    params.setServerNames(List.of(new SNIHostName(host)));
    socket.setSSLParameters(params);

    socket.startHandshake();

    X509Certificate leaf = (X509Certificate) socket.getSession().getPeerCertificates()[0];
    Instant expiry = leaf.getNotAfter().toInstant();
    long daysLeft = ChronoUnit.DAYS.between(Instant.now(), expiry);
    ...
}
```

> **A SUBTLETY WORTH MARKS**
>
> If you use Java's default trust manager, the handshake **throws an exception when the certificate is already expired or invalid** — which is precisely the case you most want to report on. Your monitoring tool would fail to monitor the exact failure it exists to catch.
>
> The fix: use a permissive trust manager *for this probe only*, so the handshake completes and you can read the certificate, then evaluate validity yourself in code. Do not disable verification globally — it must be scoped to the certificate-inspection probe, and you should say so explicitly when explaining it.

Do not check only the expiry date. A complete TLS check verifies four things:

1. **Days until `notAfter`** — the headline number.
2. **Hostname match** — does the certificate actually cover the name requested (SAN entries, wildcards)?
3. **Chain validity** — intermediate certificates expire too, and that breaks sites just as thoroughly.
4. **Not-yet-valid** — a wrongly issued certificate with a future `notBefore` is rare but real.

What gets stored

A `check_run` with `days_remaining`, the issuer name, the subject, and the exact expiry timestamp. Because this check runs daily rather than every minute, its table stays small, so you can afford to keep full history.

What goes wrong

- **Tiered warnings, not one threshold.** Alerting only at 7 days gives a team one weekend to react. Warn at 30, 14, 7, 3 and 1 days, and escalate severity as it approaches.
- **Alert fatigue.** Because the condition persists for 30 days, naive alerting would fire every single day. Alert on *threshold crossings* only — this is a specific instance of the deduplication rule in §6.5.
- **A certificate renewed automatically** should silently close the alert, which your incident engine already does for free.

5.4 Domain expiry

Why it is needed

Domain expiry is rarer than certificate expiry, and far more damaging. A lapsed domain takes down the website, all email to that domain, and anything authenticating against it. Recovery can be slow, expensive, or impossible if someone else registers it. Renewal notices go to a registrant email address that is frequently an ex-employee's inbox.

How it connects

Pull. The user types a domain name. Note that the query goes to the *registry*, not to the customer's server — this is the only check where you are talking to a third party rather than to the monitored system itself. Worth mentioning in a viva; it shows you understand what you are actually querying.

How the information is fetched

Two protocols, and you should prefer the modern one.

Protocol	How it works	Verdict for your project
RDAP	An ordinary HTTPS GET returning structured JSON. `https://rdap.org/domain/example.com` bootstraps to the right registry automatically.	**Use this first.** JSON parses reliably; no scraping.
WHOIS	A 1980s plain-text protocol on TCP port 43. Send the domain, read back free text.	Fallback only. Every registry formats differently.

```
// RDAP: expiry lives in the events array
{
  "events": [
    { "eventAction": "registration", "eventDate": "2019-04-11T08:22:15Z" },
    { "eventAction": "expiration",   "eventDate": "2027-04-11T08:22:15Z" }
  ]
}
// You want the event whose eventAction == "expiration".
```

> **BE HONEST ABOUT COVERAGE IN YOUR REPORT**
>
> RDAP support is mandatory for generic TLDs (`.com` , `.net` , `.org` …) but **varies considerably among country-code TLDs**. Since you will very likely want to monitor a `.in` domain, verify RDAP availability for it early and implement the WHOIS fallback if it is missing. Writing "RDAP where available, WHOIS fallback otherwise, with per-TLD parsing and known gaps documented" in your limitations section is a far stronger answer than pretending one protocol covers everything.

What gets stored

Expiry date, registrar name, days remaining, and the raw response (truncated) so you can debug a parsing failure later without re-querying.

What goes wrong

- **Rate limiting.** Registries will block you for aggressive querying. Once per day per domain is plenty — registration dates do not change hourly. Cache the result and never let a user configure a 1-minute interval on this check type.
- **Unparseable responses.** Record a distinct `PARSE_FAILURE` state. "I could not determine the expiry" is not the same as "the domain is expiring," and conflating them produces false alarms.
- **Auto-renew.** A domain set to auto-renew still shows a near-term expiry date, so a user may get warnings they consider noise. Offer an acknowledge or snooze action.

5.5 TCP port reachability

Why it is needed

Plenty of critical services speak no HTTP at all: PostgreSQL on 5432, Redis on 6379, SSH on 22, SMTP on 25, MongoDB on 27017. An HTTP probe cannot check any of them. TCP checks also let you verify a service *behind* the web layer — your site may be up while the database it depends on is unreachable.

How it connects

Pull. Host and port.

How the information is fetched

You attempt to open a TCP connection, and whether it succeeds *is* the answer. You send no data and immediately close.

```
try (Socket socket = new Socket()) {
    long start = System.nanoTime();
    socket.connect(new InetSocketAddress(host, port), timeoutMs);
    return ProbeResult.ok(elapsed(start), "connected");
} catch (SocketTimeoutException e) {
    return ProbeResult.fail(timeoutMs, "timed out — host unreachable or firewall dropping");
} catch (ConnectException e) {
    return ProbeResult.fail(elapsed(start), "connection refused — host up, nothing listening");
}
```

> THE DISTINCTION THAT SHOWS UNDERSTANDING
>
> **Connection refused** comes back almost instantly, because the host actively sent a TCP RST saying "nothing is listening here." The machine is alive; the service is not.
>
> **Timeout** means your packets vanished with no reply at all — typically a firewall configured to DROP rather than REJECT, or a machine that is entirely down. Ambiguous, and slow to discover.
>
> Recording these as different failure reasons rather than a generic "down" is exactly the kind of diagnostic detail that makes the tool genuinely useful.

What gets stored

Success flag, connection time in milliseconds, and the failure category. Connection latency is a useful signal on its own — a database that used to accept connections in 5 ms and now takes 800 ms is in trouble even though the check still passes.

What goes wrong

- **You may look like a port scanner.** Repeatedly opening and immediately closing connections can trigger intrusion-detection tools like fail2ban and get your IP banned. Keep intervals sane and document that the user should only monitor hosts they own.
- **Connecting is not the same as working.** A database can accept TCP connections while refusing every query. Name this limitation before your examiner does — it is precisely why the check is called *reachability*, not *health*.
- **IPv4 versus IPv6.** A host resolving to both may be reachable on one and not the other. Record which address family you actually connected to.

5.6 Multi-step synthetic transaction

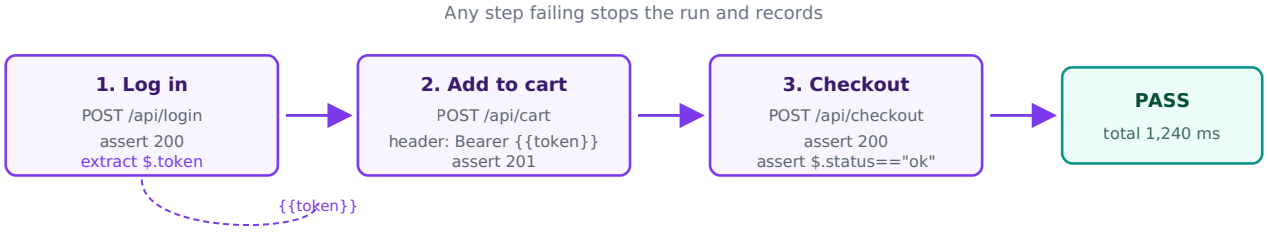
This is the feature that carries the most engineering weight in your project. Expect the deepest questions here.

Why it is needed

Everything so far answers "does the server respond?" This answers the far more valuable question: **"can a user actually accomplish something?"** Real outages routinely leave every individual endpoint returning 200 while the flow that matters — log in, add to cart, pay — is broken. A synthetic transaction is a robot customer that completes that journey every few minutes and complains when it cannot.

How it connects

Pull, but richer: instead of one URL, the user defines an ordered list of steps in your dashboard. Each step has a request, a set of assertions, and optionally instructions to extract a value for later steps to use.



which step failed — that is the diagnostic value. State (the token) flows forward between steps. This is what makes it a transaction, not three separate checks.

Figure 5.2 — A synthetic transaction carries state between steps.

How the information is fetched

You build a tiny interpreter. It walks the steps, keeping a `Map<String,String>` of extracted variables, substituting `{{name}}` placeholders into each request before sending it.

```
Map<String,String> context = new HashMap<>();
for (Step step : steps) {
    HttpRequest req = render(step, context);        // substitute {{token}} etc.
    Response res = send(req);

    for (Assertion a : step.assertions())         // status, jsonPath, contains, maxMs
        if (!a.passes(res))
            return SyntheticResult.failedAt(step, a, res);

    for (Extraction e : step.extractions())       // e.g. token ← $.data.accessToken
        context.put(e.name(), e.apply(res));
}
return SyntheticResult.passed(totalMs);
```

The steps are stored as JSON on the check's `config` column, which is exactly why §3.5 chose a JSON column — a synthetic check's configuration has no fixed shape:

```
{
  "steps": [
    { "name": "login",
      "method": "POST", "url": "https://shop.example/api/login",
      "body": { "email": "{{TEST_EMAIL}}", "password": "{{TEST_PASSWORD}}" },
      "assert": [ {"type":"status","equals":200},
                  {"type":"jsonPath","path":"$.token","exists":true} ],
      "extract": [ {"name":"token","from":"jsonPath","path":"$.token"} ] },

    { "name": "add to cart",
      "method": "POST", "url": "https://shop.example/api/cart",
      "headers": { "Authorization": "Bearer {{token}}" },
      "assert": [ {"type":"status","equals":201} ] }
  ]
}
```

What gets stored

One `check_run` for the whole transaction, plus a per-step breakdown (step name, status, latency) so the dashboard can show "step 3 of 4 failed after 2.1 s". Total duration is a valuable trend on its own — a checkout that has drifted from 900 ms to 4 s is a problem worth surfacing before it becomes an outage.

What goes wrong

Problem	How you handle it
Credentials must be stored	The check holds a real test-account password. Encrypt secret config values at rest, never return them from the API (write-only fields), and never log them. This is the most sensitive data in your entire database.
Side effects	Running a checkout flow every ten minutes creates 144 real orders a day. Use a dedicated test account and a flow that can be cleaned up, or a staging environment. Document this — it is a genuine operational concern, not a detail.

Problem	How you handle it
Flakiness	Multi-step flows fail occasionally for reasons that are not real outages. Retry the whole transaction once before recording a failure, and require consecutive failures before opening an incident (§6.4).
Cost	A four-step transaction is four requests. At a one-minute interval that is 5,760 requests a day against someone's production system. Default to a 5–15 minute interval and explain why.
Users can point it anywhere	Every SSRF concern in §7.3 applies here with extra force, because the user controls methods, headers and bodies, not just a URL.

5.7 Deploy timeline correlation

Why it is needed

When something breaks, the first question anybody asks is *"what changed?"* — because the overwhelming majority of incidents are caused by a change someone made, not by hardware spontaneously failing. Most cheap monitoring tools tell you *that* you are down. This feature is what lets yours suggest *why*.

It also transforms your product narrative. Without it you have a monitor; with it you have something that helps diagnose. That difference is worth saying out loud in a viva.

How it connects

Push. The user adds one step at the end of their CI/CD pipeline that POSTs deploy metadata to your platform, as shown in §4.3. If they deploy manually, they can call the same endpoint by hand or from a shell alias.

How the information is fetched

It is received, validated, and stored — there is no probing at all. The intelligence is entirely in the query you run afterwards:

```
-- For an incident that opened at T, was there a deploy shortly before?
SELECT d.* FROM deploy_event d
WHERE d.project_id = :projectId
  AND d.deployed_at BETWEEN :openedAt - INTERVAL '30 minutes' AND :openedAt
ORDER BY d.deployed_at DESC;
```

If that returns a row, the incident detail page can display a line that does most of a debugging session's work for free:

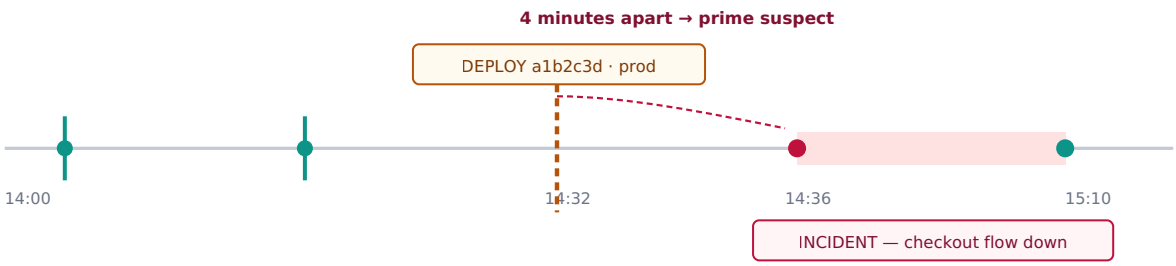


Figure 5.3 — Deploy markers overlaid on the incident timeline. The correlation does the diagnostic work.

What gets stored

Project, timestamp (server-side), commit SHA, environment, who triggered it, and optionally a link back to the pipeline run. Small table, cheap to keep forever.

What goes wrong

- **Anyone could POST a fake deploy** if the endpoint is unauthenticated. Verify an HMAC signature (§7.5).
- **Clock skew.** Store *your* receipt time as authoritative and the CI-supplied time separately. If the CI server's clock is ten minutes off, correlation silently stops working and nothing looks broken.
- **Correlation is not causation.** Your UI should say "a deploy occurred 4 minutes before this incident" and never "this deploy caused the incident." An examiner may well probe this; stating the distinction unprompted is a strong move.
- **Deploys that bypass CI** will not appear, leaving a misleading gap. Note it as a limitation.

> **A BONUS METRIC THAT COSTS YOU ALMOST NOTHING**
>
> Once you store deploys and incidents in the same database, you can compute **change failure rate** — the percentage of deploys followed by an incident within 30 minutes. This is one of the four widely cited DORA metrics used in industry to measure delivery performance. It is one SQL query on data you already have, and it is an extremely strong thing to show an examiner in the last minute of a demo.

6. The Shared Engine — Where the Real Engineering Is

Everything in Part 5 is a plug-in. This part is the socket they all plug into — and it is where examiners find the hard questions.

6.1 The scheduler, and why it is one scanner

You have (say) 200 checks, each with its own interval. Something must decide, moment to moment, which ones are due. There are two ways to build this and only one of them survives contact with reality.

The naive design: one timer per check

Give every check its own thread that sleeps for its interval and then probes. It is obvious, it is easy, and it does not scale. Each Java platform thread reserves around a megabyte of stack space, so a thousand checks means a thousand threads and roughly a gigabyte of memory doing nothing but sleeping, plus the CPU cost of the operating system switching between them. Worse, the schedule lives in memory: restart the application and every timer is lost, so you must rebuild all of them at startup and hope nothing was mid-flight.

The design you should build: one scanner, a due-time column, a worker pool

Every check row carries `next_run_at`. A single scheduled method wakes on a fixed tick and asks the database one question: *what is due?*

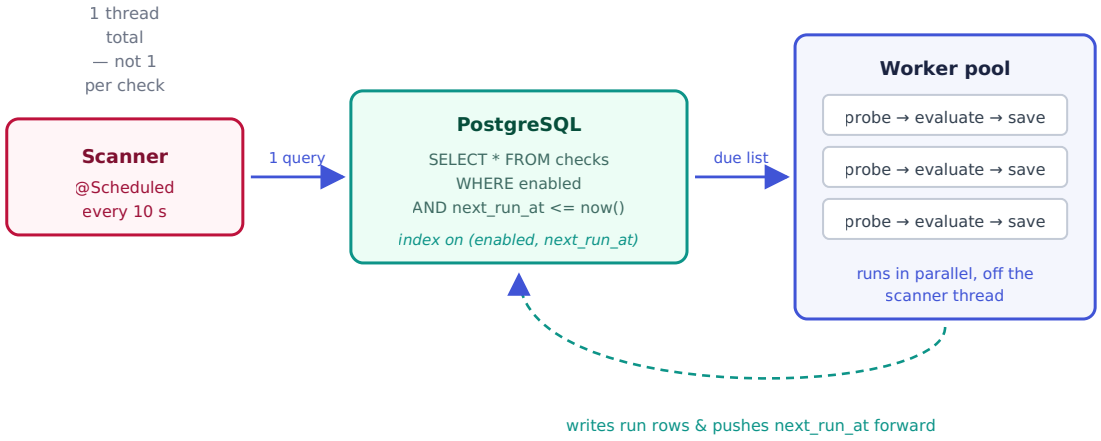


Figure 6.1 — One scanner, one indexed query, a bounded pool of workers. Memory is constant in the number of checks.

```java
@Scheduled(fixedDelay = 10_000)
public void dispatchDueChecks() {
    List<Check> due = checkRepo.findDue(Instant.now(), PageRequest.of(0, 200));

    for (Check check : due) {
        // CLAIM IT FIRST. Push next_run_at forward before probing, so the
        // next tick (10s later) does not dispatch the same check again while
        // this probe is still in flight.
        check.setNextRunAt(Instant.now().plusSeconds(check.getIntervalSeconds()));
        checkRepo.save(check);

        probeExecutor.submit(() -> checkRunner.runAndRecord(check));
    }
}
```

FOUR DESIGN POINTS AN EXAMINER CAN DIG INTO HERE

1. Claim before probing. If you update `next_run_at` only *after* the probe finishes, a probe that takes 30 seconds gets dispatched three more times by intervening ticks. Claiming first is the fix.

2. Never probe on the scanner thread. One slow site would stall every other check on the platform. Submit to a pool and return immediately.

3. Bound the batch. `LIMIT 200` stops a backlog from being loaded into memory all at once after a restart.

4. Tick faster than your finest interval. A 10-second tick can honour 60-second intervals with acceptable accuracy. It cannot honour 5-second ones — so set a minimum allowed interval and enforce it on creation.

Three refinements worth implementing

- **Virtual threads.** Probes spend almost all their time waiting on the network, not computing. Java 21 virtual threads are designed exactly for this: thousands of concurrent blocking I/O operations without thousands of operating-system threads. In Spring Boot this is close to a one-line change (`spring.threads.virtual.enabled=true`), and being able to explain *why* an I/O-bound workload is the ideal case for them is a genuinely strong answer.
- **Jitter.** If every 60-second check is created at the top of the minute, they all fire together, producing a spike every minute and idleness in between — a “thundering herd”. Add a small random offset when scheduling the next run to spread load evenly.
- **Multi-instance safety.** If you ever run two copies of the application, both scanners will find the same due checks and probe everything twice. The clean fix in PostgreSQL is `SELECT ... FOR UPDATE SKIP LOCKED` , which lets each instance claim a disjoint batch. You do not have to implement it, but **knowing it exists and naming it is one of the highest-value things you can say in a viva.**

6.2 The life of a single check, step by step

Trace one HTTP check from clock tick to email. Being able to narrate this from memory is the single best preparation for your demo.

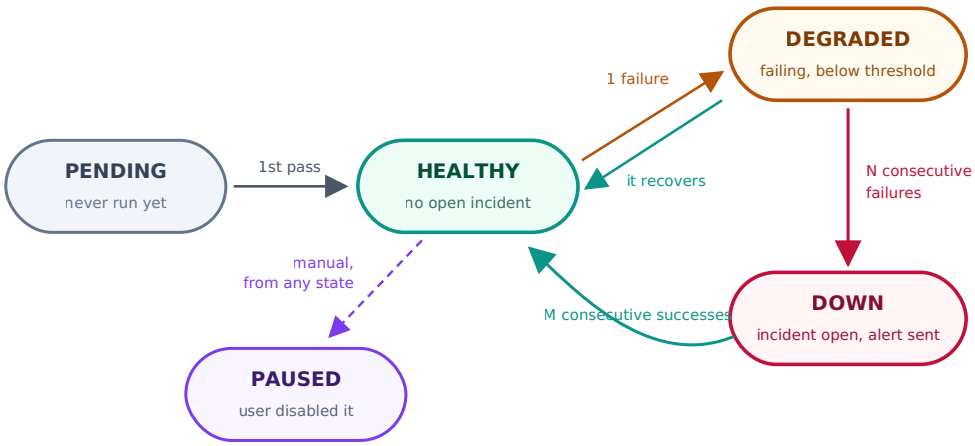
Step	What happens	Detail
1	Scanner tick	`@Scheduled` fires; one query returns the due checks.

Step	What happens	Detail
2	Claim	`next_run_at` is pushed forward immediately so nothing double-dispatches.
3	Dispatch	Handed to the worker pool; the scanner thread moves on at once.
4	Probe selection	A map lookup by `CheckType` returns the right `Probe` — no `if/else` chain.
5	Validate target	The URL is re-checked against the SSRF rules in §7.3, *at probe time, not just at creation time*.
6	Execute	The network call happens, bounded by connect and read timeouts.
7	Record the run	A `check_run` row is written with outcome, latency and detail. This always happens, pass or fail.
8	Evaluate state	Consecutive failure and success counters are updated and compared against thresholds (§6.4).
9	Transition (maybe)	If a threshold is crossed, the check's state changes and an incident is opened or closed.
10	Alert (maybe)	Only on a state *transition*, never on every failing run. Deduplicated per channel.
11	Deliver	The notifier sends; an `alert_delivery` row records what was sent and whether it succeeded.

> **STEP 7 IS NOT OPTIONAL**
>
> A very common shortcut is to record only failures, to save space. Do not. Without successful runs you cannot compute uptime percentage, you cannot draw a latency graph, and you cannot prove the checker was even running during a quiet period. Control the volume with retention (Part 8), not by discarding data you need.

6.3 The incident state machine

A **state machine** is simply a rule that says: given where you are now and what just happened, here is where you go next — and every other move is illegal. Writing it down explicitly is what stops your incident logic turning into a tangle of booleans.



DEGRADED exists so the dashboard can show trouble early without waking anybody up.

Figure 6.2 — The incident state machine. Every transition has an explicit trigger; nothing else is permitted.

An **incident** row is created the moment a check enters `DOWN` and closed the moment it leaves. That single decision is what converts a firehose of failing runs into a small number of human-meaningful events. Six hours of a site being down at one-minute intervals is 360 failed runs but exactly **one** incident — and exactly one alert, plus one recovery notice.

6.4 Hysteresis: not panicking over one blip

Hysteresis means requiring more evidence to change your mind than to keep it. Applied here: one failed probe does not mean the site is down — packets get lost, a load balancer restarts, your own network hiccups. Requiring *N consecutive* failures before declaring an outage, and *M consecutive* successes before declaring recovery, filters out the noise.

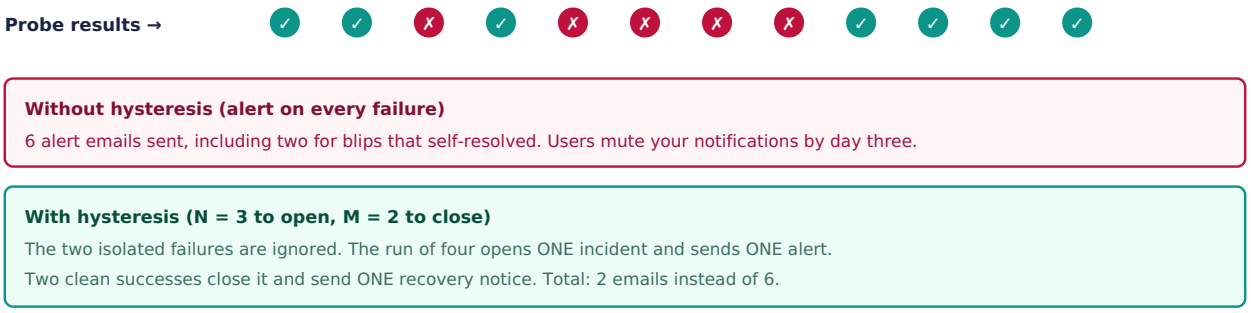


Figure 6.3 — The same probe results, with and without hysteresis.

Make N and M configurable per check, because the right values differ: a critical payment API might use N = 2 for fast detection, while a flaky third-party integration might need N = 5 to stay usable. There is a real trade-off to articulate — **higher N means fewer false alarms but slower detection**. With a 60-second interval, N = 3 means you learn about a genuine outage roughly three minutes late. Whether that is acceptable is a product decision, not a technical one, and saying so demonstrates maturity.

6.5 Alerting: deduplication, backoff, escalation

An alerting system that people mute is worse than none at all, because it creates false confidence. Three rules keep yours usable.

Rule 1 — alert on transitions, never on states

Send when a check *becomes* DOWN and when it *becomes* HEALTHY. Never send because it *is* DOWN, or a six-hour outage at a one-minute interval produces 360 emails.

Rule 2 — reminders with exponential backoff

Silence after the first alert is also wrong: a long outage should keep nagging, just not constantly. Use widening intervals — a reminder at 15 minutes, then 30, then 60, then hourly. Six hours produces roughly eight messages rather than 360, and each one carries new information ("still down, now 4 h 12 m").

Rule 3 — every send is recorded before it is attempted

The `alert_delivery` table is what makes double-sending impossible. Put a unique constraint on `(incident_id, channel, transition_type)` and the database itself guarantees at-most-once delivery per transition — even if your code has a bug and calls the notifier twice.

```
-- The database, not your code, is the guarantee.
CREATE UNIQUE INDEX uq_alert_once
    ON alert_delivery (incident_id, channel, transition_type);
```

> OPTIONAL MODULE, HIGH IMPACT
>
> **On-call escalation.** Instead of one fixed recipient, define a weekly rotation. Alert the primary; if nobody acknowledges within 10 minutes, escalate to the secondary. Acknowledgement is a single endpoint (`POST /api/incidents/{id}/ack`) plus one column. This is a small amount of code that turns a notifier into a miniature PagerDuty, and it demonstrates that you understand alerting as a *human* workflow, not just a message send.

6.6 Idempotency: the same event arriving twice

Idempotent means an operation can be performed repeatedly without changing the result beyond the first time. Pressing a lift call button twice does not summon two lifts.

This matters in your project because duplicates are not hypothetical — they are guaranteed:

- `curl --retry 3` resends when a response is slow, so a ping your server *did* process may arrive again.
- A CI pipeline re-run POSTs the same deploy event a second time.
- Your own retry logic re-attempts an alert whose success you never observed.

The general fix is a **deduplication key** plus a unique constraint. The client supplies (or you derive) a key identifying the logical event; the database rejects the second insert, and you treat that rejection as success rather than as an error:

```java
@PostMapping("/ping/{token}")
ResponseEntity<Void> ping(@PathVariable String token,
                          @RequestHeader(value = "X-Ping-Id", required = false) String pingId) {
    try {
        heartbeatService.recordPing(token, pingId, Instant.now());
    } catch (DuplicatePingException ignored) {
        // Already recorded. This is a SUCCESS, not an error — the client's
        // intent ("tell the platform I ran") has been satisfied.
    }
    return ResponseEntity.noContent().build();
}
```

> **WHY EXAMINERS LOVE THIS TOPIC**
>
> It separates students who have thought about distributed systems from those who have not. The core insight: **over an unreliable network you can never know whether a message you sent was processed — only whether you received a reply.** A missing reply may mean the request never arrived, or that it arrived and the reply was lost. Since the sender cannot distinguish these, it must retry; since it must retry, the receiver must be idempotent. Being able to state that chain of reasoning is worth a great deal.

6.7 How uptime percentage is calculated

"99.4% uptime" can mean two different things, and you should know which you implemented.

Method	Formula	Behaviour
Run-based	`successful_runs / total_runs`	Trivial to compute. But if the checker itself was down for an hour, those minutes simply do not appear, and the figure flatters you.
Time-based	`(period − downtime) / period`, where downtime is summed from incident durations	What SLAs actually mean. Requires incidents to be accurate, and needs a decision about gaps in coverage.

Implement time-based, since you already have an `incident` table with open and close timestamps. Be careful with an incident still open at the end of the window — clamp it to the window edge rather than counting it to infinity:

```sql
SELECT 1 - (
   COALESCE(SUM(
     EXTRACT(EPOCH FROM (
        LEAST(COALESCE(closed_at, now()), :windowEnd)
      - GREATEST(opened_at, :windowStart)
     ))
   ), 0) / EXTRACT(EPOCH FROM (:windowEnd - :windowStart))
) AS uptime_ratio
FROM incident
WHERE check_id = :checkId
  AND opened_at < :windowEnd
  AND COALESCE(closed_at, now()) > :windowStart;
```

For context on why the decimals matter: 99.9% allows about 43 minutes of downtime per month; 99.99% allows about 4 minutes. That difference is the entire reason companies care about the number.

6.8 The public status page

A status page is the outward-facing half of monitoring: it lets a company's customers see for themselves whether something is broken, which cuts support load dramatically during an incident.

Architecturally it is nearly free, because it is a **read-only projection of data you already have**. Four design points:

- **No authentication**, addressed by an unguessable or user-chosen slug: `/status/urbnskn`.
- **Per-check opt-in.** A `public` boolean on the check row — the internal database TCP check must not be exposed to the world.
- **Cache hard.** A status page gets its heaviest traffic during an outage, which is exactly when your database is least able to cope. Serve a snapshot recomputed every 30–60 seconds rather than querying live per visitor. This is a small detail that reads as real operational thinking.
- **Show, do not explain.** Current state per component, a 90-day history strip, and the current open incidents. Nothing more.

7. Security and Multi-Tenancy

§7.3 contains the single most impressive thing you can say about this project. Read it twice.

7.1 Two completely different kinds of authentication

Your platform has three audiences and they cannot share one authentication scheme. Getting this separation right is a design decision worth explaining.

Who is calling	Endpoints	How they authenticate	Why that choice
A logged-in human via the React dashboard	`/api/v1/**`	**JWT** obtained by logging in, sent as `Authorization: Bearer <token>`	Short-lived, carries identity and roles, and requires no server-side session store
A monitored cron job	`/ping/{token}`	**The token in the URL is the credential** — nothing else	It must work from a bare one-line `curl` inside a shell script with no login flow available
A CI/CD pipeline	`/webhook/deploy/{token}`	Project token **plus an HMAC signature** over the body	It carries a payload, so the payload itself must be proven authentic (§7.5)

In Spring Security these become separate filter chains, ordered by path, so the public paths never touch the JWT filter at all:

```
@Bean
SecurityFilterChain api(HttpSecurity http) throws Exception {
    return http
        .securityMatcher("/api/**")
        .csrf(csrf -> csrf.disable())              // stateless JWT API, no cookies
        .sessionManagement(s -> s.sessionCreationPolicy(STATELESS))
        .authorizeHttpRequests(a -> a
            .requestMatchers("/api/v1/auth/**").permitAll()
            .anyRequest().authenticated())
        .addFilterBefore(jwtFilter, UsernamePasswordAuthenticationFilter.class)
        .build();
}

@Bean
SecurityFilterChain ingest(HttpSecurity http) throws Exception {
    return http
        .securityMatcher("/ping/**", "/webhook/**", "/status/**")
        .csrf(csrf -> csrf.disable())
        .authorizeHttpRequests(a -> a.anyRequest().permitAll())  // token IS the auth
        .build();
}
```

7.2 Multi-tenancy: keeping users out of each other's data

Multi-tenancy means one running application serves many independent customers whose data must never mix. The danger is boringly simple and extremely common:

```
// BROKEN. Any logged-in user can read ANY check by guessing an id.
@GetMapping("/api/v1/checks/{id}")
CheckResponse get(@PathVariable Long id) {
    return mapper.toResponse(checkRepo.findById(id).orElseThrow());
}
```

This authenticates but does not **authorise**. The user proved who they are; nothing checked that this particular row belongs to them. This class of bug has a name — **IDOR**, Insecure Direct Object Reference — and it is one of the most frequently exploited web vulnerabilities in the wild.

The fix is to make ownership part of every query, so forgetting it becomes impossible rather than merely discouraged:

```
// CORRECT: ownership is part of the lookup itself.
@Query("""
    select c from Check c
    where c.id = :id and c.project.owner.id = :userId
""")
Optional<Check> findByIdForUser(@Param("id") Long id, @Param("userId") Long userId);
```

Two refinements that turn a correct implementation into a defensible one:

- **Return 404, not 403**, when a user requests something they do not own. A 403 confirms that check 91 exists; a 404 reveals nothing. This is called avoiding an *enumeration oracle*.
- **Write the test.** A single integration test — user A creates a check, user B requests it, expect 404 — is concrete evidence of correctness and takes ten minutes to write. Put it in your report.

7.3 SSRF — the most important security problem in this project

> **IF YOU LEARN ONE THING FROM THIS DOCUMENT, LEARN THIS**
>
> Your platform has a feature that, described plainly, is: **"a stranger types in any URL and my server fetches it for them."** That is the exact definition of a **Server-Side Request Forgery** vulnerability. It is not a bug you might accidentally introduce — it is your core feature, and you must contain it deliberately.

Why it is dangerous

Your server sits inside a network. It can reach things the attacker cannot: other containers, internal admin panels, databases bound to localhost, and — most dangerously — cloud metadata services. On many cloud providers, an HTTP request from inside a virtual machine to a special link-local address returns credentials for that machine's role. An attacker who can make your server fetch an arbitrary URL and show them the response has a straightforward path to those credentials.

So a user creating an "HTTP uptime check" for one of these is not monitoring anything — they are using your server as a proxy into your own network:

Target they enter	What they are really doing
`http://169.254.169.254/...`	Reading cloud instance metadata — potentially credentials
`http://localhost:8080/actuator/env`	Reading your own application's configuration and secrets
`http://10.0.0.5:5432`	Port-scanning your private network from inside
`file:///etc/passwd`	Reading local files, if the HTTP client accepts that scheme
`http://192.168.1.1/admin`	Reaching a router or admin panel not exposed publicly

How you defend against it

Layered, because no single check is sufficient:

1. **Scheme allowlist.** Permit only `http` and `https`. Reject `file:`, `gopher:`, `ftp:` and everything else outright.
2. **Resolve the hostname yourself, then inspect the IP.** Do not trust the string. Resolve it, and reject any address that is loopback, link-local, private (10.x, 172.16–31.x, 192.168.x), multicast, or otherwise reserved — including their IPv6 equivalents.
3. **Validate at probe time, not only at creation time.** A DNS record can be changed after the check is created — it passes validation pointing at a public address today and resolves to `127.0.0.1` tomorrow.
4. **Do not blindly follow redirects.** A public URL can 302 you straight to `169.254.169.254`. Either disable redirects, or re-run the full validation on every hop.
5. **Cap the response.** Read at most a few hundred kilobytes and store only a truncated excerpt, so a huge or infinite response cannot exhaust memory.
6. **Never return the raw body to the user.** Store a short excerpt for diagnostics; do not build a feature that echoes a fetched page back to whoever asked for it.

```
@Component
class SsrfGuard {
    void assertSafe(String rawUrl) {
        URI uri = URI.create(rawUrl);

        if (!Set.of("http", "https").contains(uri.getScheme()))
            throw new UnsafeTargetException("scheme not allowed");

        for (InetAddress addr : InetAddress.getAllByName(uri.getHost())) {
            if (addr.isLoopbackAddress()  || addr.isLinkLocalAddress()
             || addr.isSiteLocalAddress() || addr.isAnyLocalAddress()
             || addr.isMulticastAddress())
                throw new UnsafeTargetException("target resolves to a non-public address");
        }
    }
}
```

> **THE ADVANCED ANSWER, IF YOU WANT TO GO FURTHER**
>
> Even the code above has a subtle hole called **DNS rebinding**. You resolve the hostname and find a public IP, you approve it — and then the HTTP client resolves the name *again* when it actually connects, this time receiving `127.0.0.1` from an attacker-controlled DNS server with a one-second TTL. This is a genuine *time-of-check to time-of-use* race.
>
> The real fix is to connect to the **already-validated IP address** rather than re-resolving the name, while still sending the original hostname in the `Host` header and SNI. You are not obliged to implement this in a college project — but naming DNS rebinding as a known residual risk in your limitations section is a genuinely graduate-level observation.

7.4 Protecting the heartbeat token

The ping URL is a **bearer credential**: possession of it is sufficient to mark a job as healthy. Treat it accordingly.

- **Generate it with a cryptographically secure random source** — a UUIDv4 or 128+ bits from `SecureRandom`. Never a sequential ID, a hash of the check name, or anything derived from a timestamp.
- **Understand the actual risk.** An attacker who guesses a token can send false "I'm fine" pings, suppressing a real alert. That is a **denial of alerting** attack, and it is worse than it sounds: the victim believes they are being monitored when they are not.
- **Rate-limit the endpoint.** It is unauthenticated and world-reachable, which makes it the natural target for both brute-force guessing and simple flooding.
- **Make tokens regenerable**, so a token pasted into a public repository can be rotated without deleting the check and losing its history.
- **Keep them out of your logs.** A token in a URL ends up in access logs by default; mask the path segment when logging.

7.5 Verifying the deploy webhook

The deploy endpoint accepts data that will later be used to explain incidents, so it must be authentic. The standard mechanism is an **HMAC signature**: you and the sender share a secret; the sender hashes the exact request body together with that secret and sends the result in a header; you recompute it and compare.

```
boolean verify(byte[] rawBody, String headerSignature, String secret) {
    Mac mac = Mac.getInstance("HmacSHA256");
    mac.init(new SecretKeySpec(secret.getBytes(UTF_8), "HmacSHA256"));
    String expected = "sha256=" + HexFormat.of().formatHex(mac.doFinal(rawBody));

    // Constant-time comparison: a normal String.equals() returns early on the
    // first differing byte, which leaks how much of the signature was correct.
    return MessageDigest.isEqual(expected.getBytes(UTF_8),
                                 headerSignature.getBytes(UTF_8));
}
```

Two implementation details that are easy to get wrong:

- **Hash the raw bytes as received**, not a re-serialised object. If Spring parses the JSON and you re-serialise it to verify, whitespace and key ordering will differ and the signature will never match. Capture the body with a `ContentCachingRequestWrapper` or read it as a `byte[]`.
- **Use constant-time comparison.** This defends against a **timing attack**, where an attacker measures tiny differences in response time to discover a signature one byte at a time. Knowing why `MessageDigest.isEqual` is used instead of `equals` is a strong detail to have ready.

8. Data Lifecycle — How Fast Rows Pile Up

This is the part students discover two weeks before submission, when the free database fills up.

8.1 The arithmetic

Do this calculation now rather than later. One check running every 60 seconds produces:

Scope	check_run rows	At roughly 200 bytes per row
One check, one day	1,440	~0.3 MB
One check, one year	525,600	~105 MB
20 checks, one year	**10,512,000**	**~2 GB, before indexes**
20 checks, one year, with indexes	—	realistically 3–4 GB

> **WHY THIS IS NOT THEORETICAL FOR YOU**
>
> Free managed PostgreSQL tiers are commonly capped around **1 GB**. With twenty one-minute checks you would reach that in roughly four months. Your demo database will be fine; the point is that an examiner asking "what happens after a year?" deserves a real answer, and "I measured it, here is the number, here is my retention policy" is a very good one.

8.2 Retention and rollups

The standard solution is that **old data becomes less detailed rather than disappearing**. Nobody needs second-by-second results from four months ago; they need the shape of the month.

Age of data	What you keep	Used for
0–7 days	Every individual run	Debugging a live incident, precise latency graphs
7–90 days	Hourly rollups: count, failure count, avg / p95 latency	Trends, monthly uptime reports
90+ days	Daily rollups, plus every incident row kept forever	Long-term history, the 90-day status page strip

Incidents are never rolled up or deleted — they are small, few, and the most valuable rows in the database. A nightly scheduled job performs the compaction:

```java
@Scheduled(cron = "0 30 3 * * *")   // 03:30 every day, a quiet hour
public void compactOldRuns() {
    rollupRepo.aggregateHourly(Instant.now().minus(7, DAYS));
    runRepo.deleteOlderThan(Instant.now().minus(7, DAYS));
}
```

The saving is dramatic: one check at a one-minute interval produces 1,440 raw rows a day and 24 hourly rollup rows — a **60× reduction**, while preserving everything you would actually plot on a chart.

> **ONE MORE PRACTICAL MEASURE**
>
> Deleting millions of rows in a single statement locks the table and can time out. Delete in batches (say 10,000 rows at a time, in a loop with a short pause). Mentioning batched deletion unprompted signals that you have thought about running this system, not just writing it.

9. How the React Frontend Talks to Spring Boot

The frontend is a client of your API like any other. Four decisions are worth making deliberately.

9.1 They are two separate programs

React runs in the user's browser. Spring Boot runs on your server. They share nothing but the API contract — no objects, no memory, no session. Every interaction is an HTTP request carrying JSON. The React app is, from your backend's point of view, indistinguishable from `curl`.

The practical consequence people forget: **anything the frontend enforces is a suggestion, not a rule.** Disabling a button in React does not prevent anyone from calling the endpoint directly. Every validation and every ownership check must exist on the server; the frontend copy exists only to make the interface pleasant.

9.2 CORS, and how to avoid needing it

In development, React runs on `localhost:5173` and Spring Boot on `localhost:8080`. Different ports mean different **origins**, and browsers block cross-origin requests unless the server explicitly permits them — the **CORS** mechanism. You will meet this on your first day of frontend work, so configure it deliberately:

```
@Bean
CorsConfigurationSource corsConfigurationSource() {
    CorsConfiguration c = new CorsConfiguration();
    c.setAllowedOrigins(List.of("http://localhost:5173", "https://sentra.app"));
    c.setAllowedMethods(List.of("GET","POST","PUT","PATCH","DELETE"));
    c.setAllowedHeaders(List.of("Authorization","Content-Type"));
    UrlBasedCorsConfigurationSource src = new UrlBasedCorsConfigurationSource();
    src.registerCorsConfiguration("/**", c);
    return src;
}
```

> **A LEGITIMATE SIMPLIFICATION FOR DEPLOYMENT**
>
> Build the React app (`npm run build`) and copy the output into Spring Boot's `src/main/resources/static/` . Now one JAR serves both the API and the UI from a single origin: no CORS at all, one thing to deploy, one URL. For a college project this is the right call. Use the CORS config above during development, when the two run separately.

9.3 Where to keep the JWT

There are two options and neither is perfect. Know the trade-off, because it is a common viva question.

Storage	Vulnerable to	Verdict
`localStorage`	**XSS** — any injected JavaScript can read it	Simple, works everywhere. Acceptable for this project if you are careful about rendering untrusted content.
`httpOnly` **cookie**	**CSRF** — the browser attaches it automatically, so other sites can trigger authenticated requests	JavaScript cannot read it, which defeats token theft — but now you must implement CSRF protection.

The honest summary: *"I used localStorage for simplicity. It is exposed to XSS, so I mitigated by never rendering raw HTML from user input and by keeping access tokens short-lived. The production-grade alternative is an httpOnly cookie with CSRF tokens, which trades one attack class for another."* That answer is far better received than claiming either option is simply secure.

9.4 Keeping the dashboard fresh

Monitoring data changes constantly, so the dashboard must update itself. Three options, in increasing complexity:

Technique	How it works	Recommendation
Polling	The frontend re-requests every 20–30 seconds	**Start here.** Trivial, robust, entirely adequate for data that changes on a one-minute cycle.
Server-Sent Events	One long-lived HTTP connection; the server pushes updates down it	A good upgrade if you want live incident banners. Spring supports it directly with `SseEmitter`.
WebSocket	Full duplex, both sides can send at any time	Unnecessary here — your data flows one way. Adds complexity without adding capability.

Resist the temptation to poll every second. Your underlying data only changes when a probe completes, so polling faster than your shortest check interval produces load with no new information.

10. What “24×7” Actually Requires

Your platform is only a monitor if it is awake. On free hosting, that is not automatic.

10.1 The requirement

As established in §1.8, the scheduler must run continuously, independent of any user. That means the application has to live somewhere that keeps running when your laptop is closed. A monitoring platform that only works while you are demoing it does not satisfy its own premise, and an examiner is entitled to ask how you proved it.

10.2 The free-tier trap, stated precisely

> **VERIFY THIS BEFORE YOU COMMIT TO A HOST**
>
> Render’s free web services **spin down after 15 minutes without inbound traffic** and take about a minute to start again on the next request. A spun-down service is not running, so **your scheduler is not running either** — probes stop, misses go undetected, alerts are never sent. The free tier also allots 750 instance-hours per workspace per month, and free PostgreSQL databases are deleted 30 days after creation (with a short grace period to upgrade).
>
> The database expiry is the one that bites students: a database created in week 3 disappears around week 7, taking all your accumulated history with it. **Check the current terms yourself before you start** — hosting policies change, and your report should cite what you verified and when.

10.3 Options, with honest trade-offs

Option	How it behaves	Verdict
Paid instance for the final month	Always on, no spin-down, persistent database	**The clean answer.** Roughly the price of two coffees for the month that actually matters. Budget for it.
An always-free cloud VM	A small virtual machine you manage yourself; runs continuously	Genuinely free and always on, but you install Java, PostgreSQL and a service manager yourself. Good if you want the DevOps experience.
Self-pinging to stay awake	An external service requests your URL every few minutes so it never idles out	Works, but it is a workaround, not a guarantee, and it does not save you from the database expiry. Do not build your demo on it.
A spare machine at home	An old laptop or a Raspberry Pi running the JAR	Legitimate and instructive. Your home connection is the weak point, but for collecting demo evidence it is fine.

One more concrete gotcha that will cost you an evening if you meet it unprepared: **some hosting providers block outbound SMTP ports** (25, 465, 587) on free plans to prevent spam. If your alerting sends email through raw SMTP, it will simply fail there. Use an email provider’s **HTTP API** instead of SMTP — an ordinary HTTPS call, which is never blocked, and which also gives you delivery status you can record in `alert_delivery`.

10.4 Turn continuous operation into demo evidence

Because your platform runs unattended, it accumulates proof of its own value — and this is far more persuasive than a live click-through.

- **Deploy early**, even when half-built. Every week it runs is history you can show.
- **Screenshot a real multi-week uptime graph** and put it in your report. “This is 38 days of data the system collected on its own” is a much stronger claim than any live demo.
- **Keep a genuine incident.** Deliberately break something for ten minutes, let the platform detect it, alert, and recover. That timeline — detection, alert, recovery — is the single best artefact in your entire submission.
- **Monitor your own platform with itself.** Point a heartbeat check and an HTTP check at Sentra’s own `/actuator/health`. It is a satisfying thing to show, and it demonstrates you understood the product.

11. Glossary

Everything defined in one place, for revision the night before.

Term	Meaning in this project
Backpressure	What a system does when work arrives faster than it can be processed — here, a bounded queue in front of the probe pool.
Bearer token	A credential where mere possession grants access. Your heartbeat ping URL is one.
Black-box monitoring	Observing a system only from the outside, as a user would, with nothing installed on it.
CORS	The browser rule that blocks a page on one origin from calling an API on another unless the API permits it.
Dead man's switch	A mechanism triggered by the *absence* of an expected signal. The principle behind heartbeat checks.
DORA metrics	Four widely used measures of software delivery performance. Deploy correlation gives you one of them, change failure rate, almost free.
Grace period	Extra tolerance added to an expected interval before something is declared late.
HMAC	A keyed hash proving a message came from someone holding the shared secret and was not altered.
Hysteresis	Requiring more evidence to change state than to stay in it — N failures to open, M successes to close.
IDOR	Insecure Direct Object Reference: reading someone else's row by changing an ID in the URL.
Idempotent	An operation that can be repeated with no additional effect beyond the first time.
Incident	One continuous period of unhealthiness, grouping many failed runs into a single human-meaningful event.
Jitter	A small random offset added to scheduling times to stop many checks firing simultaneously.
JWT	A signed token carrying identity claims, letting the server authenticate a request without storing a session.
Multi-tenancy	One application instance serving many customers whose data must never mix.
Probe	One outbound attempt to determine whether something is healthy.
Pull / Push	Who starts the conversation: your platform reaching out, or the monitored thing calling in.
RDAP	The modern JSON-over-HTTPS replacement for WHOIS, used for domain expiry lookups.
Rollup	Replacing many detailed rows with one summary row to control storage growth.

Term	Meaning in this project
SNI	Server Name Indication — telling a server which hostname you want during the TLS handshake, so shared hosts return the right certificate.
SSRF	Server-Side Request Forgery: tricking a server into making requests on the attacker's behalf. Your central security concern.
State machine	An explicit set of states and the only legal transitions between them.
Synthetic transaction	A scripted multi-step user journey run on a schedule to verify a flow works end to end.
Thundering herd	Many scheduled tasks firing at the same instant, producing spikes. Fixed with jitter.
Timing attack	Deducing a secret from how long a comparison takes. Defeated by constant-time comparison.
Virtual threads	Lightweight Java 21 threads suited to large numbers of blocking I/O operations — exactly your probe workload.

12. Viva Questions, Rated by Depth

Forty-three questions a teacher could realistically ask, each rated for depth and each with the answer to give.

12.1 How the ratings work

The number out of 10 rates **how deep the question is** — how much genuine understanding it probes — not how hard it is to memorise an answer. A question rated 2 can be answered from your title slide. A question rated 9 cannot be answered at all unless you actually understand distributed systems.

Rating	Band	What it tests
1–3	**Basics**	Did you build this and can you describe it? Answer instantly and move on.
4–5	**Design understanding**	Do you know why the pieces are arranged this way? Most of a viva lives here.
6–7	**Engineering judgement**	Did you consider alternatives and make trade-offs knowingly?
8–10	**Deep / research-grade**	Concurrency, security and distributed-systems reasoning. Answering even two of these well changes how your project is graded.

> **HOW TO USE THIS SECTION**
>
> Do not memorise answers word for word — it is audible. Read each answer, close the document, and say it in your own words. If you cannot, go back to the Part referenced in the answer. Pay particular attention to the 8–10 band: those are the questions where a good answer is genuinely rare, so they carry disproportionate weight.

12.2 Level 1 — Basics

Q1 What does your project do? — 1 / 10

It is a unified monitoring platform. It watches websites, APIs, TCP services, TLS certificates, domain registrations and scheduled background jobs from one place, opens an incident when something fails, alerts the owner, and records deployments so you can see whether an outage started right after a code change.

Q2 Which technologies did you use and why? — 2 / 10

Java 21 with Spring Boot 4.1 for the backend, PostgreSQL for storage, React for the dashboard. Spring Boot because it provides the scheduling, security and persistence infrastructure this project needs rather than just a web layer; PostgreSQL because I need indexed time-range queries and a JSON column type; React because the dashboard is stateful and component-based.

Q3 What is a REST API? 2 / 10

A way of exposing operations over HTTP where the URL identifies a resource and the HTTP method identifies the action on it. `GET /api/checks/42` reads check 42; `DELETE /api/checks/42` removes it. My API is stateless — each request carries its own authentication and the server keeps no session between calls.

Q4 What is a “check” in your system? 2 / 10

One thing being watched. It has a type, an interval, a target, and a current state. It is the central entity — every run, every incident and every alert hangs off a check.

Q5 What is the difference between 401 and 403? 3 / 10

401 means the server does not know who you are — the token is missing, malformed or expired. 403 means it knows exactly who you are and you are not permitted. **EXTRA CREDIT** In my API I deliberately return **404** rather than 403 when a user requests a resource belonging to someone else, because a 403 would confirm that the resource exists.

Q6 What is JPA, and what does Spring Data add to it? 3 / 10

JPA is the Java specification for mapping objects to relational tables; Hibernate is the implementation. Spring Data JPA sits on top and generates repository implementations from interface method names, so `findByEnabledTrueAndNextRunAtBefore(Instant t)` becomes working SQL with no body written. I still write explicit JPQL for the queries that matter, such as the scheduler’s due-check query.

Q7 What do the three layers — controller, service, repository — each do? 3 / 10

The controller speaks HTTP and nothing else. The service holds the rules. The repository talks to the database. The reason it matters in *this* project specifically is that my scheduler calls the service directly with no HTTP request in sight — so if the rules lived in the controller, the background engine could not reuse them.

12.3 Level 2 — Design understanding

Q8 How does your platform connect to the systems it monitors? Do you install something on their servers? 4 / 10

Nothing is installed and no SSH is involved. Five of my seven features are **pull**: the user types a URL or hostname into my dashboard and my scheduler makes outbound connections on its own timetable — the monitored system never knows it exists. Two are **push**: for heartbeat and deploy tracking, the user adds a single line of `curl` to a job or pipeline they already run. **SEE** Part 4.

Q9 Why is heartbeat push while uptime is pull? Why not use one model for everything?

5 / 10

Because you cannot observe a cron job from outside. There is no port to knock on that reveals whether a backup script ran at 2 a.m. — the only entity that knows is the script itself, so it has to tell me. Conversely a website is externally observable, so pull works and requires zero setup from the user. The rule is: **pull when the state is externally visible, push when only the thing itself knows.**

Q10 Why one `check` table with a JSON column instead of a table per check type?

5 / 10

So the scheduler asks one table one question. With seven tables, "what is due?" becomes seven queries, the incident table needs seven nullable foreign keys, and adding a check type means editing the scheduler, the incident engine and the dashboard. With one table plus a JSON config column, a new check type is one class implementing the `Probe` interface and nothing else changes. **TRADE-OFF I ACCEPTED** the database can no longer validate type-specific fields, so validation moves into the application layer with a config class per type.

Q11 What is an incident, and why not just send an alert on every failed check?

5 / 10

An incident is one continuous period of unhealthiness. Without it, a six-hour outage on a one-minute check would send 360 emails and every user would mute my notifications within a week. With it, 360 failed runs become one incident, one alert, periodic reminders with widening gaps, and one recovery notice. **The incident entity is what makes the alerting usable.**

Q12 How do you stop one user reading another user's checks?

5 / 10

Ownership is part of every query, not a separate check that can be forgotten — `findByIdForUser(id, userId)` rather than `findById(id)` followed by an `if`. The vulnerability this prevents is IDOR, and I have an integration test that creates a check as user A, requests it as user B and asserts a 404.

Q13 Why JWT rather than server-side sessions?

4 / 10

Statelessness. The token carries the identity and is verified by signature, so the server holds no session state and any instance can serve any request. **HONEST TRADE-OFF** a JWT cannot be revoked before it expires without reintroducing server-side state, so I keep access tokens short-lived and use a refresh token for longer sessions.

Q14 Walk me through exactly what happens when a monitored site goes down.

5 / 10

The scheduler tick finds the check due and claims it by pushing `next_run_at` forward. A worker thread runs the HTTP probe, which times out. A `check_run` row is written with the failure category. The evaluation service increments the consecutive-failure counter; below the threshold the check goes to DEGRADED and no alert is sent. On the third consecutive failure it transitions to DOWN, an incident row opens, and the notifier sends one alert, recording an `alert_delivery` row. Reminders follow with exponential backoff. When two consecutive probes succeed, the incident closes and a recovery notice is sent. **SEE** §6.2.

Q15 Why use Flyway migrations instead of letting Hibernate create the schema? 4 / 10

`ddl-auto` is convenient but non-deterministic and cannot express data migrations, indexes or constraints I actually care about. Flyway makes the schema a versioned, reviewable sequence of SQL files that runs identically on my laptop and in production. It also means I can add the index the scheduler depends on and know it exists everywhere.

Q16 How does the public status page work, and how is it different from the dashboard? 4 / 10

It is an unauthenticated read-only projection of data I already have, addressed by a project slug. Only checks explicitly marked public appear, so an internal database check is never exposed. It is cached and recomputed every 30–60 seconds rather than queried live, because its traffic peaks precisely during an outage — when the database is least able to absorb it.

12.4 Level 3 — Engineering judgement

Q17 Why one scheduler scanning for due checks, instead of a timer per check? 7 / 10

A thread per check does not scale: each platform thread reserves around a megabyte of stack, so a thousand checks costs roughly a gigabyte of memory to do nothing but sleep, plus context-switching overhead. The schedule would also live only in memory and be lost on restart. Instead each check row carries `next_run_at`; one scheduled method runs a single indexed query for what is due and submits the work to a bounded pool. Memory is constant in the number of checks, and the schedule survives restarts because it is in the database. SEE §6.1.

Q18 What happens if a probe takes longer than the check's interval? 7 / 10

This is why I claim the check *before* probing rather than after: `next_run_at` is pushed forward the moment it is dispatched, so a probe still in flight is not dispatched again by the next tick. If I updated it only on completion, a 30-second probe on a 10-second tick would be launched three more times concurrently. I also cap the read timeout below the interval so a probe cannot outlive its own schedule slot.

Q19 What is hysteresis, why did you implement it, and what values did you choose? 6 / 10

It means requiring more evidence to change state than to keep it — N consecutive failures to open an incident, M consecutive successes to close it. Without it, one dropped packet sends an alert. I default to N=3 and M=2, configurable per check. THE TRADE-OFF TO NAME higher N means fewer false alarms but slower detection: at a 60-second interval, N=3 means I learn about a real outage about three minutes late. Whether that is acceptable is a product decision, and I made it explicit rather than accidental.

Q20 How do you calculate uptime percentage? 6 / 10

Time-based, not run-based. Run-based (successes over total runs) is easier but flatters the result, because if my own checker was down for an hour those minutes simply do not appear in the denominator. I sum incident durations over the window, clamping any still-open incident to the window edge, and subtract from the total. That is what an SLA actually means. For scale: 99.9% permits about 43 minutes of downtime a month; 99.99% permits about 4.

Q21 Why does your certificate check need a permissive trust manager? Isn't that insecure? 7 / 10

Because with Java's default trust manager the handshake throws when the certificate is already expired or invalid — the exact case the feature exists to report. My monitor would fail to monitor the thing it monitors. So for this probe alone I complete the handshake without trust validation, read the certificate, and evaluate validity myself in code. It is scoped to the inspection probe and never applied globally; every other HTTP client in the application uses full verification.

Q22 How does the synthetic transaction carry state between steps? 6 / 10

A context map lives for the duration of one run. Each step can declare extractions — for example pull `$.token` from the login response — which are written into the context, and later steps reference them as `{{token}}` placeholders that are substituted before the request is sent. That flow of state is exactly what makes it a transaction rather than three unrelated checks, and it is why a failure records *which step* failed, not just that something did.

Q23 Your synthetic check stores a real password. How do you handle that? 7 / 10

It is the most sensitive data in my database. Secret config values are encrypted at rest with a key held outside the database, the fields are write-only in the API so they are never returned in a response, and they are excluded from logs and from error messages. Operationally I also recommend a dedicated low-privilege test account rather than a real user's credentials, because a synthetic checkout flow run every ten minutes creates real side effects.

Q24 Why RDAP rather than WHOIS for domain expiry? 6 / 10

WHOIS is a plain-text protocol with no standard format — every registry lays out its response differently, so parsing is fragile screen-scraping. RDAP returns structured JSON over HTTPS with a defined events array, so I read the expiration event reliably. **LIMITATION I DOCUMENT** RDAP is mandatory for generic TLDs but coverage varies among country-code TLDs, so I keep a WHOIS fallback and record a distinct `PARSE_FAILURE` state — "I could not determine expiry" must never be reported as "expiring soon".

Q25 What happens to your database after a year of running? 6 / 10

I measured it: twenty checks at one-minute intervals produce about 10.5 million run rows a year, roughly 2 GB before indexes — which would exhaust a typical 1 GB free database tier in about four months. So I keep raw runs for 7 days, hourly rollups to 90 days and daily rollups beyond, deleting in batches to avoid long table locks. Incidents are never rolled up or deleted; they are few and they are the most valuable rows I have.

Q26 How do you test code that only runs on a schedule? 7 / 10

By never testing the annotation. `@Scheduled` is just a trigger, so I keep the method body trivial and put the logic in a service I can call directly from a test with controlled inputs. Time is injected as a `Clock` bean rather than read from `Instant.now()`, so a test can fix "now" and assert that a heartbeat 61 minutes overdue is detected while one 59 minutes overdue is not. Probes are tested against a local mock HTTP server, which also lets me simulate timeouts and malformed responses deterministically.

Q27 What stops your platform sending the same alert twice? 6 / 10

A unique constraint on `(incident_id, channel, transition_type)` in the `alert_delivery` table. The database itself guarantees at-most-once delivery per transition, so even if my code has a bug and calls the notifier twice, the second insert fails and nothing is sent. **I put the guarantee in the database rather than in application logic, because application logic is where bugs live.**

12.5 Level 4 — Deep / research-grade

Q28 Your core feature is "a user gives me any URL and my server fetches it." What is the security implication? 10 / 10

That is the textbook definition of Server-Side Request Forgery, and in my case it is not a bug — it is the product, so it has to be contained deliberately. My server sits inside a network and can reach things the user cannot: localhost, private ranges, and cloud metadata endpoints that return machine credentials to anything that asks.

My defence is layered: an allowlist of `http` and `https` schemes only; resolving the hostname myself and rejecting loopback, link-local, private and multicast addresses in both IPv4 and IPv6; **re-validating at probe time rather than only at creation time**, because DNS records can be changed afterwards; not blindly following redirects, since a public URL can 302 into the metadata range; capping response size; and never returning the fetched body to the user.

IF PRESSED FURTHER a residual risk remains — DNS rebinding, where the name resolves to a public address during validation and to 127.0.0.1 when the client actually connects. The complete fix is to connect to the already-validated IP while sending the original hostname in the Host header and SNI. I documented it as a known limitation rather than claiming it is solved.

Q29 What happens if you run two instances of your application? 9 / 10

Both schedulers would find the same due checks and probe everything twice — duplicate load on the monitored systems, duplicate rows, and potentially duplicate alerts. The alert duplication is already prevented by the unique constraint in `alert_delivery`, but the probing is not. The clean fix in PostgreSQL is `SELECT ... FOR UPDATE SKIP LOCKED` in the due-check query, so each instance atomically claims a disjoint batch and skips rows another instance holds. An alternative is a distributed lock library such as ShedLock so only one instance runs the sweep at a time, though that gives up parallelism. My current deployment is a single instance, so this is a documented scaling path rather than something I implemented.

Q30 Why must your heartbeat endpoint be idempotent? 9 / 10

Because duplicates are guaranteed, not hypothetical. I recommend `curl --retry 3` in the integration snippet, so a ping my server processed but whose response was lost will be sent again. The underlying principle is that **over an unreliable network the sender can never know whether a message was processed — only whether a reply arrived.** A missing reply could mean the request never landed, or that it landed and the reply was lost, and the sender cannot distinguish the two. Since it must therefore retry, my receiver must be idempotent. I handle it with a deduplication key and a unique constraint, and I treat a duplicate as success rather than as an error — because the client's intent has in fact been satisfied.

Q31 Who monitors the monitor? How do you know your own platform is running?

8 / 10

This is a real problem — if my platform dies, it stops alerting and its silence looks exactly like everything being healthy, which is the worst possible failure mode. A monitoring system cannot fully verify itself, because the component that would report the failure is the one that failed. My mitigations: I expose `/actuator/health` and register a heartbeat check on a free external service so something outside my infrastructure is watching me, I record a "last successful scheduler tick" timestamp and surface it on the dashboard so staleness is visible, and the status page shows when data was last refreshed rather than implying freshness. **THE GENERAL PRINCIPLE** the outermost watcher must always be external to the system it watches.

Q32 What if your own hosting provider has a network problem? Won't every check fail at once?

8 / 10

Yes, and I would alert every user about an outage that is mine, not theirs — which destroys trust in the tool faster than missing a real outage would. My mitigation is a correlation heuristic: if an unusually high proportion of all checks across all projects fail within the same window, the most probable explanation is me, so I suppress alerting and raise a platform-level notice instead. The proper industry solution is probing from multiple independent locations and only alerting when several agree, which I documented as future work since a single-region deployment cannot do it.

Q33 Why do you not trust the timestamp the client sends with a heartbeat ping?

8 / 10

Because clocks on other people's machines are routinely wrong, sometimes by hours, and a client-supplied time is also trivially forgeable — anyone could backdate a ping to hide a late job. I record arrival time from my own clock, stored in UTC, and treat any client timestamp as informational metadata only. The same reasoning applies to the deploy webhook: I store my receipt time as authoritative and the CI-supplied time separately, because if the CI server's clock is skewed by ten minutes my deploy correlation silently stops matching and absolutely nothing looks broken.

Q34 Why compare the webhook signature in constant time instead of with `equals`?

8 / 10

To defeat a timing attack. `String.equals` returns as soon as it finds a differing byte, so a comparison failing at byte 20 takes measurably longer than one failing at byte 2. An attacker who can send many requests and measure response times can discover the correct signature one byte at a time, reducing an infeasible brute force to a linear search. `MessageDigest.isEqual` compares every byte regardless of mismatches, so the timing carries no information. The related detail is that I hash the **raw request bytes**, not a re-serialised object — re-serialising changes whitespace and key order, so the signature would never match.

Q35 How would this design change at 100,000 checks?

9 / 10

Four things break in a predictable order. **Scheduling** first: one scanner cannot dispatch 100,000 checks per minute, so I would shard by check ID across several workers claiming disjoint batches with `SKIP LOCKED`. **Write throughput** second: that is roughly 1,700 run-inserts per second, so I would batch inserts and move the run table to a time-series design — in PostgreSQL, native partitioning by time, which also makes retention a matter of dropping a partition instead of deleting rows. **Threads** third: virtual threads handle the I/O-bound concurrency, since the workload is almost entirely waiting on sockets. **Storage** fourth: rollups stop being an optimisation and become mandatory.

THE HONEST PART I have not built at that scale. What I have done is make sure nothing in my current design *prevents* those changes — the due-check query is already a single indexed statement, and claiming is already a discrete step that can become an atomic claim.

Q36 Your deploy correlation shows a deploy before an incident. Does that prove the deploy caused it?

8 / 10

No, and my UI is deliberately worded to avoid claiming it does — it says “a deploy occurred 4 minutes before this incident,” never “this deploy caused it.” Correlation is evidence, not proof: the deploy may be unrelated, or both may share a third cause such as a traffic spike that triggered both an autoscaling deploy and the failure. The feature’s value is that it narrows the search space at the moment when a human is under pressure and reasoning badly. **WORTH ADDING** because I store deploys and incidents in one database, I can also compute change failure rate — the proportion of deploys followed by an incident — which is one of the four DORA delivery metrics, and that is an aggregate statistic rather than a causal claim about any single deploy.

Q37 Why is your probe pool sized the way it is, and why does thread count not track CPU count here?

8 / 10

The usual rule — threads roughly equal to cores — applies to CPU-bound work. My probes are almost entirely **blocking network I/O**: a thread issues a request and then waits, consuming no CPU. So the pool can be far larger than the core count, and the real limit is memory and socket descriptors rather than CPU. This is precisely the workload Java 21 virtual threads were designed for, since they are not bound to operating-system threads while parked on I/O. I also bound the queue in front of the pool, so that if probes are arriving faster than they complete I fail fast and log it rather than accumulating an unbounded backlog that eventually exhausts memory.

Q38 A user sets a check to 10-second intervals against someone else’s website. What are the implications?

8 / 10

Two distinct problems. Technically, my platform becomes an unwitting load generator — 8,640 requests a day from one check, and nothing stops a user creating fifty of them, at which point I am running a small denial-of-service attack from my own IP address, which will get me blocked or blacklisted. Ethically and legally, my platform should only be pointed at systems the user is authorised to monitor. My mitigations are a minimum interval enforced server-side per check type, a per-project cap on total request rate, an identifying `User-Agent` so site operators can see who I am and contact me, and terms stating that users must own or be authorised to monitor their targets. This is a genuine abuse vector, and monitoring products that ignore it get their IP ranges blacklisted.

Q39 Someone guesses a heartbeat token. What can they actually do?

8 / 10

They can send false healthy pings, which suppresses a genuine alert. That is a **denial-of-alerting** attack, and it is more dangerous than it first sounds because the victim believes they are being monitored when they are not — a silent failure is worse than a visible one. Mitigations: tokens are 128-bit values from `SecureRandom`, never sequential or derived; the endpoint is rate-limited since it is unauthenticated and world-reachable; tokens can be rotated without losing check history, because they end up pasted into scripts and sometimes committed to public repositories; and I mask the token path segment in access logs so it does not leak through log files.

Q40 Why do you write a row for successful runs too? Isn't that wasteful? 7 / 10

It is the difference between a system that can prove things and one that cannot. Without successful runs I cannot compute uptime percentage, cannot draw a latency trend, and cannot demonstrate that the checker was even running during a quiet period — silence would be ambiguous between "everything was fine" and "my scheduler was dead." The volume problem is real, so I solve it with retention and rollups rather than by discarding data I need. **Throwing away the data is optimising the wrong variable.**

Q41 Your TCP check succeeds but the database is refusing every query. Is your check wrong? 8 / 10

The check is correct but the question it answers is narrower than the question the user cares about. A TCP check verifies *reachability* — something is listening — not *health*. That is why I named it reachability rather than health, and it is a real limitation of black-box monitoring generally: I can see that a port accepts connections, but not what happens behind it. The layered answer is that reachability, HTTP status and synthetic transactions form a ladder of increasing fidelity and increasing cost, and a user should pick the rung that matches what they need. The genuine fix for the database case is a synthetic check that runs a trivial query through an application endpoint.

Q42 What is the weakest part of your system? 8 / 10

Single-region, single-instance deployment. It creates a single point of failure — if my host has a problem, monitoring stops entirely and, worse, stops silently. It also means I cannot distinguish "the site is down" from "the network path between me and the site is down," because I only have one vantage point. The fix is multiple probe locations with quorum-based alerting, which is architecturally straightforward — probes are already stateless — but needs infrastructure beyond this project's scope. **WHY ANSWER THIS WAY** naming a real weakness with a specific remedy reads as engineering maturity; claiming there are no weaknesses reads as not having looked.

Q43 What would you do differently if you started again? 8 / 10

Answer this from actual experience, not modesty. A strong shape: *"I built the HTTP check first and only extracted the* `Probe` *interface when I added the second type, so my first probe carried assumptions that leaked into the abstraction and I had to rework it. Next time I would design the interface against two check types from the start. I would also add the retention job in week two instead of week nine — I discovered the storage growth problem by arithmetic rather than by hitting it, and I would rather the system were correct by construction than corrected later."*

12.6 The five questions you must not fumble

If time is short, prepare these five. They are the ones most likely to be asked and most damaging to answer badly.

Q	Question	Why it matters
Q8	How does it connect to what it monitors?	The most natural question about the whole project. Answer it in two sentences: five pull, two push, nothing installed.
Q14	Walk me through a site going down.	Tests whether you understand your own system end to end. Practise saying it out loud.
Q17	Why one scheduler rather than a thread per check?	The clearest demonstration that you made an engineering decision rather than following a tutorial.
Q28	The SSRF question.	The single highest-value answer available to you. Very few student projects can answer it at all.
Q42	What is the weakest part?	Examiners ask this to see whether you evaluated your own work. Have a real answer with a real remedy ready.

A CLOSING NOTE ON HOW TO ANSWER

When you do not know something, say so and then say what you would do to find out. “I have not implemented that — my understanding is it would need X, and I would verify by Y” scores far better than a confident guess, because examiners are testing your judgement, not your recall. And whenever you can, name the trade-off you accepted. **Engineering is the business of choosing between imperfect options knowingly, and showing that you chose is what separates a project from an exercise.**